

Full length article

CPDetector: Automated detection of excessive permissions in Chrome extensions

Xiaoyu Cheng^{a, *}, Zhi Wang^{a, *}, Wanpeng Li^b, Wei Zeng^c, Xin Yang^a^a College of Cyber Science, Nankai University, Key Laboratory of Data and Intelligent System Security, Ministry of Education, Tianjin, China^b Department of Computer Science, University of Liverpool, Liverpool, United Kingdom^c University of Aberdeen, Aberdeen, United Kingdom

ARTICLE INFO

Keywords:

Security risks
Chrome extensions
CPDetector
Chrome Web Store
PermissionGuard

ABSTRACT

Extensions are specialized components designed to enhance the functionality of web browsers. However, Chrome's permission management system lacks fine-grained control, which creates a widespread pattern of extensions declaring permissions for which no corresponding usage is detected in their source code. Such permission-API mismatches may pose significant security risks, including unauthorized data access and user tracking. To address this problem, we present *CPDetector*, a static triage framework that flags Chrome extensions with permission-API mismatches. *CPDetector* analyzes whether an extension requests more permissions than the APIs it is detected to use, by extracting its declared permissions and comparing them to the APIs statically detected in its source code. Using *CPDetector*, we analyzed 199,058 extensions from the Chrome Web Store and flagged 49.3% (98,072) of them as permission-API mismatch candidates, a triage signal of potential permission over-declaration. We also introduce *PermissionGuard*, a browser extension that flags Chrome extensions carrying a permission-API mismatch when users install or use them, helping them make more informed security decisions.

1. Introduction

Modern web browsers allow browser extensions to expand their functionality to enhance the user experience. The Chrome Web Store (CWS) provides a diverse range of extensions that offer various features, such as ad blocking, news aggregation, and UI customization (Liu et al., 2012).

In browser extension development, the permissions requested by developers and the APIs they interact with are crucial for enabling an extension's functionality (Developers, 2024a; Google Chrome Developers, 2024). Developers must declare these permissions in the `manifest.json` file to access the necessary APIs for their intended features. For example, to use the `chrome.cookies` and `chrome.history` APIs, developers must first request the `cookies` and `history` permissions (Developers, 2024, 2025). Granting such permissions allows extensions to retrieve sensitive user data, including browsing history and cookies. This creates significant privacy and security risks, as malicious or mismatch-flagged extensions could exploit these permissions for unauthorized data access (Arshad et al., 2017; Jagpal et al., 2015; Kapravelos et al., 2014; Pantelaio et al., 2020; Wang et al.,

2018; Somé, 2019; Fass et al., 2021; Yu et al., 2023; Sjösten et al., 2017; Starov and Nikiforakis, 2017).

Security mechanisms such as privilege separation, the principle of least privilege, and isolation techniques form the foundation of Chrome extension security (Barth et al., 2010). However, Chrome extension's permission management system lacks fine-grained control, and its auditing process does not always detect mismatch-flagged extensions. One major limitation of the current permission system is its inability to enforce precise access controls. For example, if an extension needs to check a user's login status on a subscription-based website, it must request the `cookies` permission. Additionally, to interact with multiple sites, it may require `<all_urls>`. However, this combination of broad permissions grants the extension access to all user cookies, including those protected by the `HTTPOnly` flag, across every visited website. This inability to restrict access based on specific contexts makes it difficult to differentiate between legitimate functionality and excessive privilege, increasing the risk of unauthorized data collection and security breaches.

When a Chrome extension is updated, Google's handling of the update depends on the type of permissions requested. If the update

* Corresponding author.

E-mail addresses: 1120230325@mail.nankai.edu.cn (X. Cheng), zwang@nankai.edu.cn (Z. Wang), Wanpeng.Li@liverpool.ac.uk (W. Li), w.zeng.24@abdn.ac.uk (W. Zeng), yx@mail.nankai.edu.cn (X. Yang).<https://doi.org/10.1016/j.cose.2026.105173>

Received 26 September 2025; Received in revised form 5 September 2026; Accepted 15 September 2026

Available online 18 September 2026

0167-4048/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

includes new permissions that trigger warnings, such as `accessibilityFeatures.modify`, Chrome's system will notify users and require explicit approval before proceeding with the update. In this scenario, the extension will be temporarily disabled until the user accepts the new permissions, at which point the update can continue. Conversely, if the update does not include any new permissions that trigger warnings, the update will proceed silently. In such cases, the browser will automatically download and install the new version without user intervention. Despite these safeguards, attackers have occasionally exploited vulnerabilities in the review process to distribute malicious extensions. For example, the extension "PDF Toolbox" (ID: `bahogceckganpcobcdgmoidngedmfo`) dynamically loads a remote configuration from serasearchtop.com using highly obfuscated code and injects malicious scripts into all webpages. It exploits CWS's silent update mechanism to evade detection for 18 months. By maintaining a consistent permission declaration, delaying activation by 24 h, and selectively responding based on geographic location, the extension circumvents security reviews and continuously executes unauthorized code injection (Palant, 2023).

Not only malicious extensions but also benign ones may exhibit mismatch-flagged behavior (Picazo-Sanchez et al., 2022; Heule et al., 2015). While existing security architectures and review mechanisms offer partial user protection, these vulnerabilities reveal critical limitations in Chrome extension permission management (Chrome, 2023a). Therefore, we conducted an in-depth study on permission-related issues in Chrome extensions, focusing specifically on the problem of permission-API mismatches introduced by developers.

CPDetector does not claim that every flagged extension is malicious. Instead, four categories of extensions emerge from a permission-API mismatch. The first category covers extensions that request broad permissions due to careless engineering, such as copying a boilerplate manifest from a template project. The second category covers extensions that reserve permissions for planned features not yet shipped in the analyzed version. The third category covers extensions that contain dormant code paths. In these extensions, a permission is declared and a corresponding API call exists in the codebase. However, the call is gated behind a condition, a remote configuration flag, or a future update. The fourth category covers extensions that are already malicious and exploit the permission immediately. Static analysis alone cannot separate these four categories with certainty.

What *CPDetector* provides is a triage signal: an extension with unused, security-sensitive permissions deserves closer scrutiny, regardless of which category it eventually falls into. The PDF Toolbox case discussed earlier illustrates why this triage matters. That extension kept its permission declaration unchanged across 18 months while activating malicious behavior only after a delay and only for specific geographic regions. A permission-API mismatch detected at any point during that window would have flagged the extension before its dormant logic activated.

To detect excessive permission requests, we propose **CPDetector** (Chrome Permission Detector), a static triage framework. It flags permission-API mismatch candidates by comparing declared permissions against the APIs detected in an extension's source code. Furthermore, to enhance user security and privacy while using Chrome extensions, we integrated *CPDetector* into a dedicated Chrome extension called **PermissionGuard**. This tool proactively flags potential permission-API mismatches for users, helping them make more informed decisions about their installed extensions.

Prior studies on extension permission-API mismatch, most notably (Picazo-Sanchez et al., 2022), examined only API-level permissions and predated the mandatory Manifest V3 transition. *CPDetector* advances this line of work along three dimensions that prior art does not address. First, it operates on a 2024–2025 dataset of 199,058 extensions, making it the first large-scale permission-API mismatch study conducted after Google enforced Manifest V3 adoption industry-wide. Second, it introduces *dual-dimensional detection*: API permissions are

evaluated through a permission-API mapping table. Host permissions are assessed through a separate three-criterion procedure that accounts for Chrome's mandatory coupling of sensitive APIs (e.g., cookies) with broad host-permission declarations. This coupling forces developers to declare `<all_urls>` even when they need access to a single domain, a structural constraint that has no counterpart in Android or any prior Chrome permission study. Third, *CPDetector* produces a four-level risk taxonomy grounded in three security criteria: privacy sensitivity, Chrome's native warning triggers, and granting risk. This taxonomy converts detection output into actionable triage rather than a binary verdict. Furthermore, we integrated *CPDetector* into a dedicated Chrome extension named **PermissionGuard**. This tool is designed to boost user security and privacy by actively warning users if installed extensions show a permission-API mismatch.

The contribution points of this paper are as follows:

1. We propose *CPDetector*, a static triage framework that flags permission-API mismatch candidates in Chrome extensions. By extracting declared permissions and identifying the APIs detected in the extension's source code, *CPDetector* flags discrepancies as potential permission-API mismatch signals.
2. Using *CPDetector*, we flagged 49.3% (98,072 out of 199,058) extensions as permission-API mismatch candidates. This proportion reveals a widespread pattern in Chrome extension development. Many extensions declare permissions for which *CPDetector* detects no corresponding usage in the source code.
3. To protect user security and privacy, we have developed a Chrome extension called *PermissionGuard*, which flags installed extensions that show a permission-API mismatch, helping them identify potential risks.

Section 2 presents the relevant background on Chrome extensions and their permission system. Section 3 analyzes the problems in the current Chrome extension permission system. Section 4 describes *CPDetector*, which flags permission-API mismatches in Chrome extensions. Section 5 reports the experimental evaluation. Section 6 describes *PermissionGuard*, the browser extension built on this detection mechanism. Section 7 reviews related work. Section 8 discusses the limitations of the proposed method. Section 9 concludes the paper.

2. Background

2.1. Chrome extension

A Chrome extension is architecturally composed of four core elements: background scripts (service workers), content scripts, Web Accessible Resources (WARs), and UI pages, as shown in Fig. 1.

Background Script: The core functionality of a Chrome extension is implemented in background scripts (or service workers in Manifest V3), which run independently of the web page's lifecycle (Chrome, 2023b). These scripts have access to most Chrome extension APIs (e.g., `chrome.tabs`), provided the necessary permissions are explicitly declared in the `manifest.json` file.

Content Script: A content script is a JavaScript file injected by an extension into a web page, where it runs within the context of that page (Chrome, 2012). While it has access to the page's standard DOM APIs, it operates in an isolated world — a separate JavaScript environment that prevents interference with the page's own scripts and other extensions' content scripts. Content scripts have limited access to Chrome extension APIs, with only a small subset available — such as `connect()`, `sendMessage()`, `getManifest()`, `getURL()`, `id`, and event listeners like `onConnect` and `onMessage`. These APIs are primarily intended for communication with background scripts. Importantly, they do not require explicit permission declarations, ensuring that content scripts run with minimal privileges. This design reduces security risks, such as privilege escalation through compromised web pages.

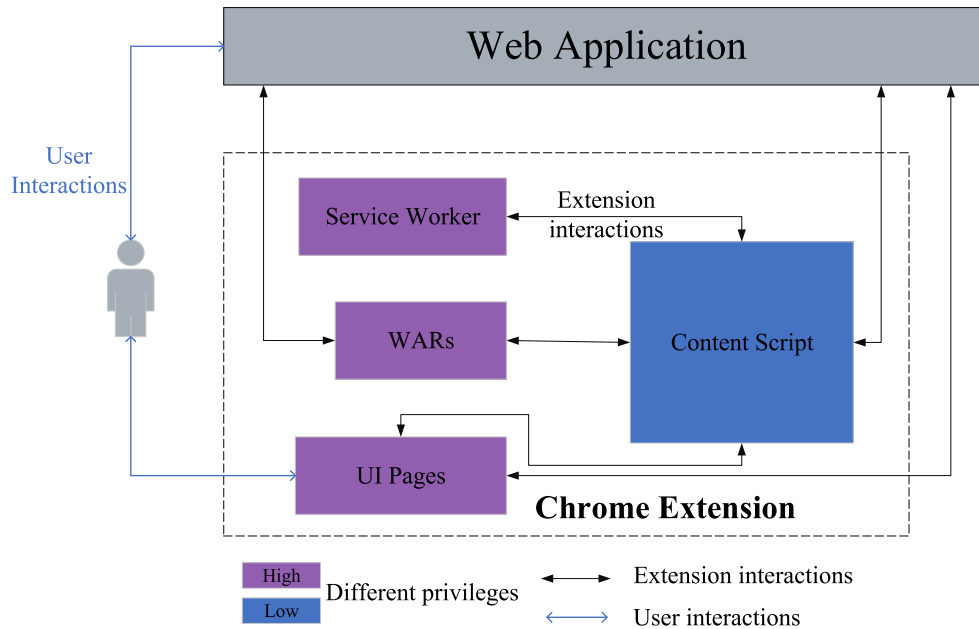


Fig. 1. The architecture of chrome extension.

Table 1
Chrome extension script permissions.

JS type	Accessible APIs	DOM access	JS access	Cross-Origin requests
Content script	Limited access (dom, i18n, storage, runtime.connect(), runtime.getManifest(), runtime.getURL(), runtime.id, runtime.onConnect, runtime.onMessage, runtime.sendMessage())	Yes	Yes	No
Popup script	Full access (excluding devtools APIs)	Yes (can access its own popup's DOM)	Yes	Yes
Background script	Full access (excluding devtools APIs)	No (needs communication with content script to access DOM)	Yes	Yes

UI Pages: Extensions may introduce user interface elements or option pages, allowing individuals to tailor the functionality according to their preferences (Developers, 2012b).

WARs: Additionally, an extension can provide Web Accessible Resources (WARs), enabling these files to be reachable by both web pages and other extensions.

2.2. Chrome extension security architecture

While browser extensions enhance functionality, they also introduce significant security risks. Traditionally, extensions are granted excessive privileges by default, despite most features requiring only minimal access to browser functionalities, creating opportunities for attackers to exploit vulnerable extensions. To mitigate these threats, Barth et al. (2010) proposed a browser extension architecture based on the principles of least privilege, privilege separation, and strong isolation. Their system was designed to minimize the potential damage an attacker could inflict through a compromised or malicious extension. This architecture laid the foundation for the modern extension security model used by major browsers. It was first implemented in Google Chrome and has since been adopted by other leading browsers, including Firefox and Microsoft Edge.

2.2.1. Security architecture

The Google Chrome extension system implements a three-tiered defense architecture based on the principles of least privilege, privilege separation, and strong isolation (Barth et al., 2010). Its core mechanisms operate as follows:

- **Least Privilege:** An extension must explicitly declare its required permissions in the manifest file at installation time. The browser grants access only to the permissions listed in the declaration. This limits the potential impact of a compromised or buggy extension.
- **Privilege Separation:** Content scripts interact with web pages through restricted DOM APIs. They do not have direct access to powerful extension APIs and can only communicate with the background scripts using structured JSON messages. The background script holds the declared API permissions and interacts with content scripts via message passing or XMLHttpRequests (Carlini et al., 2012).
- **Strong Isolation:** Chrome enforces multiple layers of isolation. First, through the *isolated worlds* mechanism, content scripts and web page scripts run in separate JavaScript contexts even though they share the same DOM (Islam, 0000). Second, Chrome assigns dedicated processes to the extension core and native binaries, isolating them from the web rendering process. This design helps mitigate the risk of exploiting browser-level vulnerabilities. Furthermore, each extension is assigned a unique origin based on a public key (e.g., chrome-extension://{publicKeyId}/). This ensures strict separation between extensions and enables seamless updates while preserving access to persistent storage.

2.2.2. The permission system

The principle of least privilege in Chrome extensions is enforced through a permission system that requires extensions to explicitly

declare their required privileges in a manifest file. This declarative permission model categorizes access requests into five distinct types to balance functionality and security:

- **permissions**: Specifies core API permissions required at installation time. Changes to these permissions may trigger security warnings during updates.
- **optional_permissions**: Lists additional permissions that can be requested dynamically at runtime. These require explicit user approval when invoked.
- **content_scripts.matches**: Defines the URL patterns where content scripts will be injected, determining which web-pages the extension can interact with.
- **host_permissions**: Declares the domains the extension can access. Modifying these permissions may trigger a warning.
- **optional_host_permissions**: Allows the extension to request domain-specific access at runtime, giving users more control over when and where access is granted.

Developers must explicitly declare the permissions depicted in [Tables A.13](#) and [A.14](#) within the manifest file to utilize corresponding extension APIs. [Listing 1](#) provides an example of how various permissions are declared in the manifest.

Listing 1: Manifest permissions Configuration

```
{
  "name": "Example Extension",
  "version": "1.0",
  "permissions": [
    "notifications",
    "clipboardRead"
  ],
  "optional_permissions": [
    "history"
  ],
  "host_permissions": [
    "https://*.analytics.com/*"
  ],
  "optional_host_permissions": [
    "https://*.usercontent.com/*"
  ],
  "manifest_version": 3
}
```

[Table 1](#) compares permissions across Chrome extension scripts: content scripts have limited API access and DOM interaction, while pop-up/background scripts support full API privileges (excluding devtools) and cross-origin requests.

3. Problems

Although Chrome's browser extension architecture introduces privilege separation by allowing developers to request access to specific APIs and host resources, several limitations still exist. First, it lacks a user-facing interface that allows users to selectively revoke individual permissions granted to an extension after installation. Second, the permission model does not support fine-grained control over specific API access. As a result, extensions can request broader permissions than necessary, potentially gaining access to resources far beyond their actual requirements. This undermines the principle of least privilege and increases the risk of abuse or exploitation.

A permission-API mismatch is not itself an attack. It is an expanded attack surface. Three mechanisms turn this surface into realized harm. First, update-driven escalation exploits the silent update channel described in [Section 3.3](#). An extension ships with broad but unused permissions and accumulates a user base under that benign profile.

A later update activates the dormant API calls without triggering a new permission warning. Second, supply-chain compromise targets the gap directly. If an attacker takes over a developer account or a third-party library, they will obtain all permissions declared by the original extension. These include unused permissions, allowing the attacker to call APIs that the original legitimate code never used. Third, runtime-loaded logic, such as remotely fetched configuration or dynamically injected scripts, can call permission-gated APIs that never appear as static strings in the shipped bundle. In each mechanism, the permission was granted at install time and the user never sees a second prompt. The size of the gap between declared and used permissions sets the ceiling on what such a transition can reach.

Two documented cases illustrate how this gap converts into harm. The PDF Toolbox extension (ID: bahogceckgcanpcoabdcgmoidngedmf) kept a stable permission declaration for 18 months while its remotely loaded configuration stayed dormant, then activated script injection selectively by geographic region ([Palant, 2023](#)). No new permission prompt accompanied this activation, because the permission had already been granted at install time. A separate case involved an extension impersonating the well-known ad-blocking tool Adblock Plus. By obfuscating its name and exploiting the silent-update channel for updates that introduce no new warning-triggering permissions, this extension bypassed Chrome's review process and accumulated over 37,000 downloads, after which it covertly injected advertisements through tab hijacking ([OSCHINA, 2023](#)). Both cases share one structural feature: the capability used for harm was present in the manifest long before the harmful behavior appeared. This is the pattern *CPDetector* is designed to surface, not a claim that every flagged extension will follow it.

3.1. Coarse-grained permission model

A critical flaw stems from Chrome's requirement to bind sensitive APIs (e.g., cookies, webRequest, tabs, and declarativeNetRequest) with broad host permissions. As stated in Chrome's official documentation ([Chrome Extensions, 2024](#)), these APIs require developers to declare `host_permissions` for interacting with URLs. For example, the cookies API requires host permissions to access cookies on specific domains. However, since host permissions operate at the extension-wide level, declaring `<all_urls>` inadvertently grants unrestricted cookie access to all websites.

In the case of the grammar checking extension Grammarly ([Chrome, 2025](#)), in order to maintain a user login state, the extension needs to use the cookies API to access authentication tokens stored in its own domain (e.g., `.grammarly.com`). However, Chrome's permissions system dictates that when using the cookies API, developers must specify `host_permissions` that cover all cookie-interacting domains ([Chrome Extensions, 2024](#)). The key issue is that even if Grammarly only needs to access cookies for its own service domain (`.grammarly.com`), declaring this limited scope in `host_permissions` would prevent the extension from sending user text data to multiple back-end services, thus blocking its text-analysis feature entirely. As a result, Grammarly was forced to declare broad host permissions (`http:///*/*`, `https:///*/*`) at the global level. This coarse-grained coupling between the cookies API and host permissions automatically grants the extension read and write access to cookies on every website the user visits. This includes sensitive domains such as online banking sites. Yet the extension has no legitimate need to manipulate cookies outside its own backend service.

3.2. Lack of user-controlled permission management

The permission management model for Chrome extensions follows an "all-or-nothing" authorization approach, where users must either accept all requested permissions or choose not to install or use the extension. This model introduces two key issues:

- **Forced Acceptance of Broad Permissions:** Users are required to approve all permissions declared by an extension at the time of installation, regardless of how sensitive or broad they may be. For example, Grammarly requests both the `http://*/*`, `https://*/*` host permission (to analyze text inputs across any website) and access to the cookies API (to maintain authentication with its backend domain, such as `*.grammarly.com`). While these permissions are necessary for the extension's functionality, granting them also gives the extension access to cookies on unrelated sites, including sensitive domains like online banking platforms—posing a privacy and security risk.
- **Limited User Control Over API Permissions:** While Chrome now allows users to configure host permissions — letting them specify which websites an extension can access — it still does not provide similar control over API-level permissions. This means users cannot selectively enable or disable access to sensitive APIs such as cookies, even if they are uncomfortable with the level of access being granted. For instance, if a user wants Grammarly to access its backend but not read cookies from banking websites, there is currently no way to enforce this through Chrome's permission settings.

3.3. Silent permission escalation through updates

When a Chrome extension is updated, users are only notified if the new version requests permissions that fall into the “warning” categories. The permissions marked with a value of 1 in the “Warning” column of Table 12 indicate those capable of triggering alert notifications. If additional permissions are added that do not trigger a warning, the update proceeds silently without user notification. Moreover, developers may exploit this by requesting a broader set of warning permissions in the initial version, triggering a one-time approval, and then silently using those permissions in future updates without further user consent. This behavior creates a loophole that can undermine user trust and the principle of least privilege.

4. Permission-API mismatch detection: CPDetector

As discussed in Section 3, Chrome's permission management system offers limited control to users, often encouraging developers to request broad sets of permissions — many of which may never be used. To investigate the extent of this issue across the Chrome extension ecosystem, we developed *CPDetector* (see Fig. 2), a static triage framework that flags CWS extensions carrying a permission-API mismatch. *CPDetector* operates in three stages: In the first phase, *CPDetector* collects metadata and CRX files for all publicly available extensions from the CWS. In the second stage, the permission information and extended API information required for mismatch detection are extracted and stored in the corresponding database. In the third stage, *CPDetector* performs static analysis on each extension to compare the declared permissions in the `manifest.json` file against the actual API usage in the extension's source code. If an extension declares permissions for which no corresponding API usage is detected by *CPDetector*'s static analysis, it is flagged as a mismatch candidate and recorded in a dedicated database. Building on this pipeline, we additionally develop a Chrome extension that leverages the database of flagged extensions. This extension notifies users when an installed extension carries a flagged permission-API mismatch, helping them make more informed decisions about their privacy and security.

4.1. Data collection

To investigate the prevalence of permission-API mismatch in the Chrome extension ecosystem, it is essential to collect a comprehensive dataset of available extensions from the CWS. To facilitate this,

we designed the Extension Collector module as part of the *CPDetector* framework. This module is responsible for crawling and storing data about extensions and is composed of three core components: the Collector Engine, a Metadata Database, and a File Archive.

- **Collector Engine:** This component automates the process of crawling the CWS, retrieving detailed metadata for each publicly available extension. The collected information includes the extension ID, version number, and number of users etc. The Collector Engine also downloads the extension package in CRX format for further analysis.
- **Metadata Database:** All metadata retrieved by the Collector Engine is stored in a structured SQLite database. Each record stores an extension's ID, version number, manifest version (V2 or V3), declared permissions, host_permissions, and content_scripts fields as serialized JSON strings. This schema directly feeds the two downstream detection stages. In the Permission Extraction stage (Section 4.2), the *Declared Permission Extractor* queries this database by extension ID and version to retrieve the raw permission fields. It then parses the permissions field for API permissions and, for Manifest V3 extensions, the host_permissions field separately. The *Used Permission Extractor*, introduced alongside the *Declared Permission Extractor* in the Permission Extraction stage (Section 4.2), operates directly on the unpacked CRX source files stored in the File Archive, scanning all JavaScript files within each extension to identify API calls. In the Permission-API Mismatch Detection stage, the *API Permission Mismatch Extractor* then joins the declared permissions retrieved from this database with those API call records. For each extension, it checks whether every declared permission maps to at least one detected API call, using the permission-API correspondence tables (Tables A.13 and A.14). An extension is flagged as a mismatch candidate if even one declared permission has no corresponding API call in the codebase. Such a mismatch indicates a potential permission-API mismatch signal, though not a definitive verdict. The *Host Permission Mismatch Extractor* applies the three-criterion procedure to the host_permissions field from the same database, comparing declared wildcard patterns against the domains the extension actually contacts at the source-code level.
- **File Archive:** The File Archive is responsible for storing the raw CRX files of each extension. These files are later unpacked and analyzed in subsequent phases of the *CPDetector* framework, such as static analysis for permission misuse. The archive ensures that all original files are preserved for reproducibility and future inspection.

4.2. Permission extraction

Before an extension can invoke specific Chrome extension APIs, its developers must explicitly declare the required permissions in the `manifest.json` file. To identify whether an extension is mismatch-flagged, i.e., declaring permissions for which no corresponding usage is detected in static analysis, we must first extract both the declared and used permissions. To facilitate this analysis, we designed the Permission Extraction module within the *CPDetector* framework. This module consists of two main components: *Declared Permission Extractor* and *Used Permission Extractor*.

In Section 2.2.2, we introduced five permission types in Chrome extensions. Our detection framework specifically focuses on two core permission categories: permissions and host_permissions. This selective approach is based on three key considerations. First, our primary research objective centers on identifying systematic permission-API mismatch patterns within extension core functionalities. Required permissions represent essential security declarations for basic operations, making them critical indicators for potential permission misuse detection. Second, optional permissions (including optional_

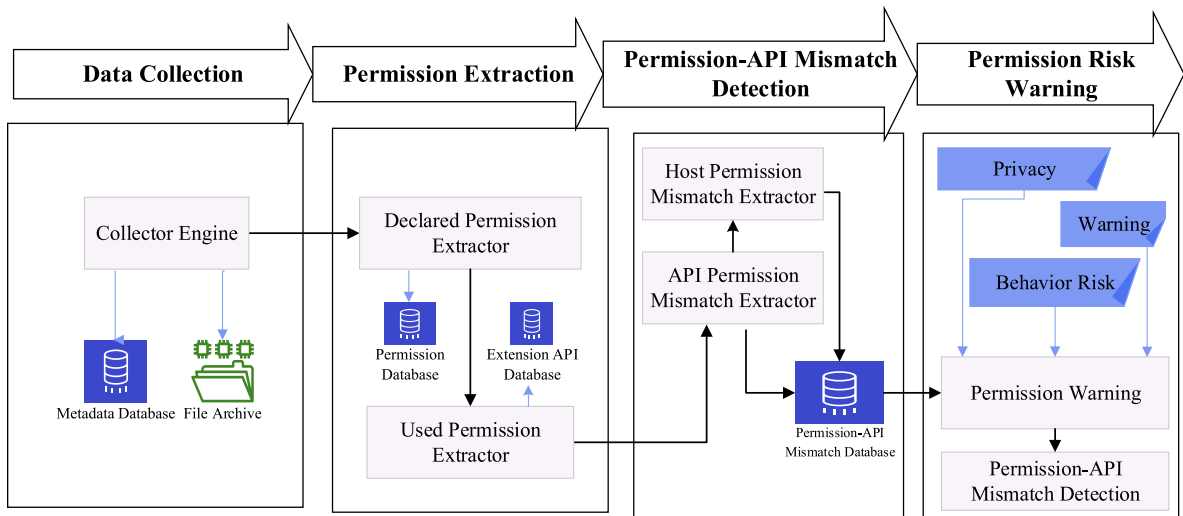


Fig. 2. Overview of CPDetector.

permissions and optional_host_permissions) require dynamic user interaction during runtime, presenting inherent limitations for static code analysis. Third, content script permissions (content_scripts.matches) operate within isolated security contexts, primarily governing webpage interactions rather than core extension functionalities. This targeted analysis aligns with our research goal of developing a lightweight yet effective detection mechanism for systematic permission-API mismatch, as opposed to conducting comprehensive permission analysis requiring dynamic runtime monitoring.

4.2.1. Declared permission extractor

This component focuses on identifying all permissions explicitly declared by the extension. It parses and analyzes the extension's manifest.json file to extract both API and host permissions. This provides a complete view of the privileges the extension requests at installation time.

Due to the coexistence of Manifest V2 and V3 extensions in the CWS, the framework implements different ways of extracting permissions based on the difference in manifest versions. For Manifest V2 extension, the system parses the manifest.json file in a structured way and extracts API permissions and host permissions from the unified permissions field simultaneously. For the Manifest V3 extension, the system implements a two-field separation extraction strategy, obtaining API permissions permissions and host permissions host_permissions separately. The extracted permission information is persisted to a structured database system in a uniform schema compatible with version heterogeneity.

Tables A.13 and A.14 presents the canonical mapping between permissions and their corresponding APIs as defined in the Chrome extension documentation (Developers, 2024b,a, 2012a). The API permissions extracted by The Declared Permission Extractor widget are based on the Permission lists of these two tables.

4.2.2. Used permission extractor

This component is responsible for identifying all permissions for which a corresponding API call is detected in the extension's source code. It performs static code analysis on all the extension's JavaScript files to detect API calls and behaviors that correspond to specific Chrome extension permissions. This includes scanning for usage patterns of host-permission-bound APIs such as chrome.tabs, chrome.cookies, chrome.declarativeNetRequest, etc.

When extracting extension APIs, the framework uses the Tool (2025) library for lexical analysis to extract identifiers. These

identifiers are then matched against a predefined list of extension APIs to reconstruct the sequence of API calls and count how many times each API is called. CPDetector extracts extension API information based on a predefined API list established in Chrome's official documentation (Developers, 2024a). This predefined list serves as the foundation for API identification and analysis in the detection process. Finally, the extracted API sequences, API call counts, extension IDs, version numbers, and other information are stored in the API database. When detecting mismatch-flagged extensions, it is necessary to query the associated APIs and permissions of the extension using both the extension ID and version number. CPDetector stores the extracted permission information and extended API information into their respective corresponding databases for subsequent permission-API mismatch detection.

We also attempted to identify the range of URLs to which the extension's source code directs requests, in order to assess whether there is an issue of permission-API mismatch in the host permissions granted to the extension. In investigating the scope of host permissions actively utilized by the extension, we employed regular expression matching to detect the following URL patterns in the code: (1) fetch() API calls; (2) XMLHttpRequest requests; (3) URL parameters in Chrome APIs (such as url and matches); (4) src, href, and action attributes in HTML tags; (5) WebSocket connections.

4.3. Permission-API mismatch detection

An extension can invoke an extension API only if it has explicitly requested the necessary permission and the user has granted it. This permission-based access control mechanism is fundamental to Chrome's security architecture, ensuring that extensions operate within explicitly authorized boundaries. However, many extensions declare permissions for which CPDetector detects no corresponding usage, or declare host-permission scopes broader than what the detected code paths require. This discrepancy creates a mismatch between the permissions granted and those the source code shows as used. Many extensions therefore carry permissions with no detected use in their source code. In permission-API mismatch detection, we focus on permission-API mismatch in core API permissions and host permissions. Therefore, we divide this module into two components: API Permission Mismatch Extractor and Host Permission Mismatch Extractor.

4.3.1. API permission mismatch extractor

This component is responsible for detecting permission-API mismatches in the core API permissions declared by Chrome extensions. To

detect mismatch-flagged extensions, our methodology systematically compares declared permissions in the manifest file against the API invocation patterns identified through static source code analysis. The detection workflow comprises four phases:

- Establishing the permission-API mapping baseline using authoritative documentation references (see Tables A.13 and A.14);
- Extending static analysis to identify both manifest-declared permissions and API signatures detected in the source code;
- Conducting differential analysis to detect discrepancies between permission declarations and API utilization;
- Persisting mismatch cases with their metadata (extension ID, version, unmatched permissions) in dual-format storage (SQLite database and JSON files) to support heterogeneous application scenarios. This structured detection output enables downstream applications to generate user alerts for installed extensions flagged with a permission-API mismatch.

4.3.2. Host permission mismatch extractor

This component is responsible for detecting permission-API mismatch of host permissions for Chrome extensions. To systematically evaluate the scope of permission requests in Chrome extensions, our analysis specifically focuses on four prevalent host permission patterns that indicate broad resource access: “http://*/*”, “https://*/*”, “*://*/*”, “<all_urls>”.

In the detection of mismatch-flagged host permissions in extensions, our analysis focuses on the host-level dimension rather than protocol or path specifications. This approach is motivated by two key considerations. First, developers typically declare access permissions at the domain level rather than specifying individual paths during extension development. Second, the protocol dimension holds limited significance for detection since modern browsers predominantly utilize HTTPS and HTTP protocols. Consequently, host-level analysis enables a more conservative and precise evaluation of excessive permission granting in browser extensions.

Following the extraction of declared and utilized host permissions by the *Declared Permission Extractor* and *Used Permission Extractor* (as described in Section 4.2), the determination of permission-API mismatch host permission is subsequently based on three critical criteria as follows.

For an extension e , let $D(e)$ denote the set of declared host match patterns collected from permissions and host_permissions. Let $A(e)$ denote the set of URL patterns recovered by the source-level scan of Section 4.2: `fetch()` calls, `XMLHttpRequest` calls, URL-valued Chrome API parameters, `src/href/action` attributes, and WebSocket connections. Let $S(e)$ denote the specific manifest-defined scopes, namely the non-wildcard entries of `content_scripts[]`. matches. Let $B(e)$ denote the broad manifest-defined scopes, namely wildcard entries of `content_scripts[]`. matches, such as `<all_urls>`.

A reduction function $\text{dom}(\cdot)$ maps a pattern or a URL to a domain symbol. It maps each of the four broad forms, `<all_urls>`, `http://*/*`, `https://*/*`, and `*://*/*`, to a single symbol $\star$. A coverage predicate $\text{cov}(p, o)$ tests whether a declared pattern p admits an observed domain o . The predicate holds for every o when p is broad. It holds for o equal to or a subdomain of h when p takes the form $\star.h$. It requires exact equality otherwise.

Three sets follow from these definitions:

$$H_D(e) = \text{dom}(D(e)) \quad (1)$$

$$H_O(e) = \text{dom}(A(e)) \cup \text{dom}(S(e)) \cup \text{dom}(B(e)) \quad (2)$$

$$H_U(e) = \{ \text{dom}(p) : p \in D(e), \text{dom}(p) \neq \star, \nexists o \in H_O(e). \text{cov}(p, o) \} \quad (3)$$

$H_D(e)$ is the declared domain set. $H_O(e)$ is the observed domain set. $H_U(e)$ is the unmatched domain set: the declared domains for which the analysis finds no corroborating evidence.

- **Criterion 1 (no declaration).** If $H_D(e) = \emptyset$, the extension declares no host permission. *CPDetector* does not flag it.

$$O(e) = 0 \quad \text{if } H_D(e) = \emptyset \quad (4)$$

- **Criterion 2 (broad access corroborated).** If $\star \in H_D(e)$ and $\star \in H_O(e)$, *CPDetector* does not flag the extension. $H_O(e)$ absorbs $\text{dom}(B(e))$. A wildcard `content_scripts` scope alone satisfies this criterion, because the browser injects the content script into every page the pattern covers. This is realized broad access, not an unused declaration. For an extension that declares only specific domains, the corresponding pass condition is $H_U(e) = \emptyset$.

$$O(e) = 0 \quad \text{if } \star \in H_D(e) \wedge \star \in H_O(e) \quad (5)$$

- **Criterion 3 (broad declaration, no broad evidence).** If $\star \in H_D(e)$ and $\star \notin H_O(e)$, the decision rests on $|H_O(e) \setminus \{\star\}|$, the count of distinct domains carrying evidence, compared against a threshold t ($t = 10$ in the reported results). Three outcomes follow. *CPDetector* does not flag the extension when the count reaches t . *CPDetector* flags the extension when the count falls below t and the extension declares no permission tied to a legitimate dynamic-access mechanism. *CPDetector* reports the extension as *undetermined* when the count falls below t but the extension declares such a permission; static analysis cannot adjudicate this case. $R(e)$ denotes the set of mechanism labels from Table 4 that the extension carries.

$$O(e) = \begin{cases} 0 & \text{if } |H_O(e) \setminus \{\star\}| \geq t \\ U & \text{if } |H_O(e) \setminus \{\star\}| < t \wedge R(e) \neq \emptyset \\ 1 & \text{otherwise} \end{cases} \quad (6)$$

The undetermined outcome is reported, not resolved. The procedure therefore yields an interval rather than a point estimate. The lower bound counts only $O(e) = 1$. The upper bound counts $O(e) = 1$ together with $O(e) = U$. Table 5 isolates the contribution of each revision to this interval.

Two permissions receive separate treatment rather than a default classification. `activeTab` grants access only to the tab a user just clicked; it narrows scope instead of broadening it, so *CPDetector* does not let it satisfy Criterion 3 by default. `cookies`, `webRequest`, and `proxy` sit on the opposite side: Chrome forces these APIs to couple with broad host permissions, the pattern the Grammarly example in Section 3 illustrates. *CPDetector* additionally reports the flagged count under two alternative readings of Criterion 3, one that lets `activeTab` explain a wildcard declaration and one that lets the three coupled APIs explain it. Neither alternative enters the main result reported in Table 7; Table 5 reports both alongside the main result for comparison.

With *API Permission Mismatch Extractor* and *Host Permission Mismatch Extractor*, information about extensions flagged as mismatch candidates, such as extension ID and extension version number, is stored in the database. We save the results through sqlite database and json files respectively to suit different application context needs.

5. Evaluation

5.1. Dataset

The experimental environment consists of 62 GB of memory, Ubuntu 20.04.1 operating system, an Intel(R) Xeon(R) Bronze 3206R CPU @ 1.90 GHz, and Python 3.11. As of May 2025, we have collected a total of 199,058 extension datasets with extension ID and version number as unique fields. We collected two types of extension data. One type consists of archived extension files, stored in a folder. The other type includes extension metadata, stored in a database.

Table 2

Most frequently mismatched permissions, occurrence counts, and share of all mismatches.

Permission	Occurrence	Share of mismatches
activeTab	49,634	25.11% (49,634/197,646)
scripting	34,611	17.51% (34,611/197,646)
storage	20,327	10.28% (20,327/197,646)
unlimitedStorage	14,692	7.43% (14,692/197,646)
webNavigation	9317	4.71% (9,317/197,646)
tabs	9184	4.65% (9,184/197,646)
notifications	5158	2.61% (5,158/197,646)
contextMenus	4979	2.52% (4,979/197,646)
cookies	4787	2.42% (4,787/197,646)
alarms	4335	2.19% (4,335/197,646)

5.2. Mismatch share across permissions

We counted, for each permission, how many times it appeared among the 197,646 recorded permission-API mismatches. This count fixes each permission's share of the mismatch pool. A larger share indicates that mismatches involving that permission account for a higher proportion of the flagged pool. This analysis helps us more comprehensively examine the matched permissions of Chrome extensions.

$$s_i = \frac{n_i}{N}, \quad (7)$$

where:

- s_i is permission i 's share of the total mismatch pool.
- n_i denotes the number of mismatches for the i th API.
- N is the total number of mismatches across all APIs, given by:

$$N = \sum_{i=1}^k n_i, \quad (8)$$

where k is the total number of distinct APIs.

5.3. Results of CPDetector

5.3.1. Results of API permissions

Our static analysis flagged 49.3% (98,072 out of 199,058) extensions as permission-API mismatch candidates. Each flagged extension declared at least one permission with no matching API call detected in its source code. A mismatch is a triage signal, not a confirmed finding. Token-based matching can miss calls built through dynamic string concatenation, since the API name never appears as a literal token in the source. This proportion still points to a real pattern. Developers often declare permissions as functional safeguards, beyond what the visible code path needs. This practice runs counter to the principle of least privilege.

We also quantified each permission's share of the mismatch pool for the most and least commonly flagged permissions, as presented in [Tables 2, 3](#) and [Figs. A.6, A.7](#). This share divides each permission's occurrence count by the total number of mismatches across all permissions, 197,646. A larger share places a permission closer to the center of the flagged pool. The `fileBrowserHandler` permission carries the smallest share, 0.3‰ (6 out of 197,646), as [Table 3](#) shows. This low share tracks how rarely the permission gets flagged at all.

Among the 197,646 recorded mismatches, `activeTab` accounts for the largest share, 25.11% (49,634 instances), and `scripting` follows at 17.51% (34,611 instances), as [Table 2](#) reports. This share measures each permission's weight inside the flagged pool. The MV3 transition replaced the MV2 pattern of `activeTab` combined with `tabs.executeScript`, and many MV3 developers now declare `scripting` as a precaution.

Table 3

Least frequently mismatched permissions, occurrence counts, and share of all mismatches.

Permission	Occurrence	Share of mismatches
<code>fileBrowserHandler</code>	6	0.3‰(6/197,646)
<code>certificateProvider</code>	4	0.2‰(4/197,646)
<code>dns</code>	4	0.2‰(4/197,646)
<code>audio</code>	4	0.2‰(4/197,646)
<code>enterprise.networkingAttributes</code>	3	0.2‰(3/197,646)
<code>processes</code>	2	0.1‰(2/197,646)
<code>enterprise.hardwarePlatform</code>	2	0.1‰(2/197,646)
<code>printing</code>	2	0.1‰(2/197,646)
<code>accessibilityFeatures.read</code>	1	0.1‰(1/197,646)
<code>fileSystemProvider</code>	1	0.1‰(1/197,646)

[Tables 2 and 3](#) together show that `activeTab` appears most often among the flagged permissions, 49,634 occurrences, a 25.11% share of the mismatch pool. `fileSystemProvider` appears least often, a single occurrence.

We also conducted a further comparison of results from detecting mismatch issues based on abstract syntax tree (AST) analysis and found that 46.5% (92,481 out of 199,058) of extensions exhibited mismatch. In contrast, the method described in this paper detected 49.3%. We believe that both approaches can effectively detect mismatch. The AST-based results exhibit higher accuracy but lower runtime efficiency. In contrast, the method proposed in this paper is more efficient and convenient, though it results in a slight increase in the false positive rate.

[Picazo-Sanchez et al. \(2022\)](#) reported a mismatch-flagged rate of 59.2% among 136,018 browser extensions in December 2019. Their abstract states a lower figure, 39.8% compliance, equivalent to 60.2% mismatch-flagged, but this figure spans a mixed corpus of 170,927 items: browser extensions, apps, and themes combined. Themes carry a compliance rate of 99.9%, which pulls the mixed-corpus figure away from the browser-extension-only rate reported in their [Table 4](#). We therefore compare against the browser-extension-specific rate, 59.2%, since our own corpus contains browser extensions exclusively.

Our dataset yields a mismatch rate of 49.3% for 2024–2025, a decline of 9.9 percentage points from the 2019 baseline. This decline does not establish that Manifest V3 improved permission hygiene. [Section 5.3.5](#) reports the opposite pattern within our own dataset: MV3 extensions carry a higher mismatch rate, 54.62%, than MV2 extensions, 36.62%. The two studies also differ in detection method, association rule mining versus token-based API matching, and in manifest-version composition, since MV3 did not exist as a mandatory format in 2019. These differences block a direct causal reading of the decline. The data support a narrower claim. Permission over-declaration persists near 50% five years after the first large-scale measurement. The platform-level transition to MV3 has not closed this gap.

We also applied *CPDetector* to 1429 extensions previously removed from the CWS by Google. Among them, 408 (28.6%) were flagged as mismatch candidates. This rate is lower than the 49.3% observed in the live dataset. The gap carries a clear implication. Google removes extensions primarily for policy violations, malware distribution, or deceptive behavior. Permission-API mismatch is not a primary removal trigger. As a result, many mismatch-flagged extensions remain on the store undetected, while removed extensions can be benign. The 28.6% rate in the removed corpus confirms this: permission-API mismatch is an ecosystem-wide problem, not a marker specific to removed extensions. Breaking down by manifest version, MV2 and MV3 extensions show nearly identical rates (27.4% vs. 29.1%), consistent with findings in the live dataset. `activeTab` remains the most over-declared permission, accounting for 244 of 606 total mismatches (40.3%).

Table 4

Permissions mapped to legitimate dynamic or manifest-based host-access mechanisms. *Carrying* counts extensions declaring at least one permission in the row. *Undet.* counts extensions that additionally lack source-level evidence and therefore enter the undetermined class of Criterion 3.

Declared permission	Mechanism	Carrying	Undet.
activeTab	User-selected page	34,847	2107
scripting	Runtime-generated target	27,938	3891
declarativeNetRequest, declarativeNetRequestWithHostAccess, declarativeNetRequestFeedback	Ad blocker	8,387	1195
accessibilityFeatures, accessibilityFeatures.read, accessibilityFeatures.modify	Accessibility tool	67	0

Counts are over $n = 129,953$ extensions. An extension carrying two mechanism permissions contributes to two rows.

Table 5

Ablation of the revised host-permission procedure at $t = 10$ ($n = 129,953$). Each row changes exactly one element relative to the row above it.

Configuration	Flagged	Rate
M0: as originally reported	33,998	26.16%
M1: + domain reduction $\text{dom}(\cdot)$	33,912	26.10%
M2: + broad manifest scopes in H_O (Criterion 2)	12,731	9.80%
M3: + specific manifest scopes in H_O (Criterion 3)	11,889	9.15%
M4: + dynamic-access mechanisms (flagged only)	7,378	5.68%
M4: undetermined	4,511	3.47%
M4: upper bound	11,889	9.15%

5.3.2. Results of host permissions

We searched the same 199,058 extensions for four broad host-permission patterns, `http://*/*`, `https://*/*`, `*/*/*/*`, and `<all_urls>`. After removing records that neither declared one of these four patterns nor exhibited any host-permission usage, 129,953 extensions remained for analysis, matching the sample used throughout Section 4.3.

We first report the baseline outcome, computed with $H_O(e)$ restricted to source-level evidence only, that is, $\text{dom}(A(e))$ from the fetch, XMLHttpRequest, Chrome API parameter, HTML attribute, and Web-Socket signals of Section 4.2. Under this baseline, 33,998 of 129,953 extensions (26.16%) are flagged. Table 5 labels this outcome *M0*.

Section 4.3 extends $H_O(e)$ with two manifest-level evidence sets, $S(e)$ and $B(e)$, to answer whether a declared wildcard host permission corresponds to an access channel the extension genuinely exercises. Table 5 traces the consequence of this extension step by step. Adding broad manifest scopes, a wildcard `content_scripts` match or a `declarativeNetRequest` rule with no domain condition, to $H_O(e)$ (*M2*) drops the flagged count from 33,998 to 12,731 (9.80%). This single step accounts for the largest correction in the pipeline. An extension declaring `<all_urls>` and injecting a content script into every page satisfies Criterion 2 by definition: the browser already executes that script on every site the permission covers, so the declaration is not idle. The *M0* procedure missed this evidence because it looked only for fetch and XHR calls, not for injection scope declared in the manifest itself.

Adding the specific manifest scopes, named content-script targets and rule-scoped DNR domains, to $H_O(e)$ (*M3*) lowers the count further, to 11,889 (9.15%). This step captures extensions that inject content scripts into a bounded but numerous list of named sites rather than every page: the named list now counts toward $|H_O(e) \setminus \{\star\}|$ in Criterion 3, and several extensions cross the $t = 10$ domain threshold on this evidence alone.

The remaining 11,889 extensions split under Criterion 3's third branch (*M4*). An extension enters this branch when $|H_O(e) \setminus \{\star\}|$ still falls below t but the extension declares a permission mapped to a legitimate dynamic-access mechanism in Table 4. Of the 11,889, 7378 (5.68% of 129,953) declare no such permission and remain flagged. The other 4511 (3.47%) are reported as *undetermined*: static analysis cannot separate a genuinely unmatched extension from one whose scripting,

DNR, or accessibility permission is legitimately gated behind a runtime condition this analysis cannot resolve. The revised procedure therefore reports an interval rather than a single figure: a lower bound of 5.68% and an upper bound of 9.15% at $t = 10$, in place of the 26.16% baseline.

Table 4 decomposes this exemption layer by permission. `activeTab` is carried by 34,847 extensions, of which 2107 lack source-level evidence and enter the undetermined class. `scripting` is carried by 27,938 extensions, with 3891 entering the undetermined class. The `declarativeNetRequest` family is carried by 8387 extensions, with 1195 undetermined. `accessibilityFeatures` is carried by only 67 extensions, none of which enter the undetermined class, a sample too small to move the overall estimate. Summed across the four rows, 51,353 of 129,953 extensions (39.52%) carry at least one exemption-mapped permission, yet only 4511 (3.47%) actually lack matching evidence and enter the undetermined class. Carrying an exemption-mapped permission does not excuse an extension by default; the permission excuses the declaration only when the extension also shows no other evidence of the access it claims.

Table 6 breaks down the baseline (*M0*) flagged set by declared pattern. `<all_urls>` accounts for 47.5% (see Fig. 3) of the flagged extensions (16,136 of 33,998), the largest single group, while the source-level scan recovered an average of only 2 domains for these extensions (max 9). The Avg and Max columns in Table 6 report $|\text{dom}(A(e)) \setminus \{\star\}|$ only, the source-level domain count that feeds the *M0* baseline; they exclude the manifest-level terms $S(e)$ and $B(e)$ introduced in *M2* and *M3*. A sizable share of the extensions counted in this table therefore carry a wildcard content-script scope not visible in these two columns, which is exactly the evidence *M2* recovers and which explains why the `<all_urls>` group shrinks most sharply once manifest evidence enters $H_O(e)$.

Our analysis also revealed that seven historical versions (3.110.0 to 3.116.1.1) of the same extension (ID: pachckjkeccfpdpbpmfolblod-fkgbhl) consistently declared full-site permissions `<all_urls>`, yet the source-level scan recovered only nine specific host domains across every version. This discrepancy is consistent with developers declaring excessive permissions as a preemptive strategy for potential future requirements, since seven consecutive releases left the declaration untouched while the observable domain footprint stayed fixed.

5.3.3. Threshold sensitivity

The domain-count threshold t in Criterion 3 acts on $|H_O(e) \setminus \{\star\}|$, as defined in Section 4.3. We reran the full revised pipeline, *M0* through *M4* of Table 5, under seven alternative values: 5, 15, 20, 30, 50, 80, and 100. Table 7 reports the baseline rate and the resulting interval for all 129,953 extensions.

The baseline curve behaves as reported before revision. The flagged rate rises with t , then slows. Moving from $t = 10$ to $t = 15$ adds 0.92 percentage points. Moving from $t = 15$ to $t = 100$ adds only 1.58 points more. The curve flattens past $t = 20$.

The revised procedure inherits this shape but adds a second question: does the interval itself stay stable, or does it widen without bound? The data answer this directly. The interval spans 3.47 percentage points

Table 6
Analysis of flagged host-permission patterns (baseline procedure, $t = 10$).

Host_Permission_Pattern	Proportion	Avg	Max	Max_Extension (Version)
["<all_urls>"]	47.5% (16,136/33,998)	2	9	pachckjkecffpdphbpmfolblodfkgbhl.3.116.1.1
["https://*/*"]	5.9% (1990/33,998)	2	9	jnfnoaobkkicomefifaafaeofielap.1.3
["http://*/*"]	0.7% (242/33,998)	1	9	nolkleckgikphcknllccijejhfmccchk.1.0.3
["https://*/*", "http://*/*"]	24.6% (8353/33,998)	2	9	fkgmkeceolkfghibcciefboghdcmbpj.0.0.0.3
["*://*/*"]	15.9% (5394/33,998)	2	9	gidejehfgombmkfflghejpnclbgfkagj.4.8.2
Other	5.4% (1883/33,998)	2	9	gghdahjgeomeiohiecmfmdomeighih.2.1.0

Avg = mean number of source-level domains, $|\text{dom}(A(e)) \setminus \{\star\}|$; Max = maximum of the same quantity across the row.

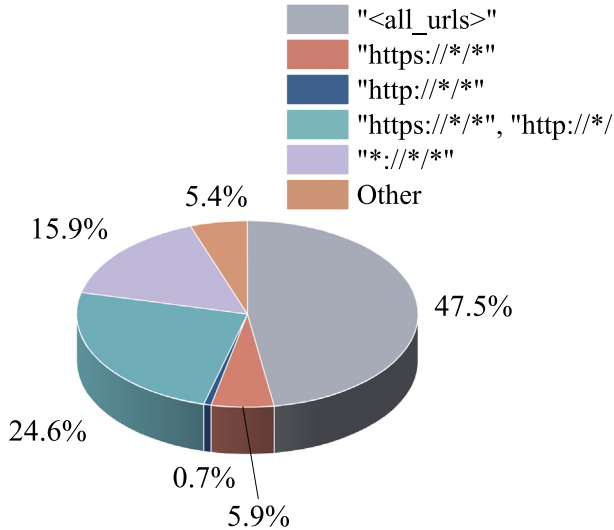


Fig. 3. Prevalence distribution of flagged host-permission patterns (baseline procedure, $t = 10$).

Table 7
Interval sensitivity of the host permission-API mismatch estimate across domain-count thresholds ($n = 129,953$).

Threshold t	M0 baseline	M3 upper bound	M4 lower bound	Interval width
5	22.76%	7.95%	4.98%	2.97 pp
10	26.16%	9.15%	5.68%	3.47 pp
15	27.08%	9.61%	5.94%	3.67 pp
20	27.66%	9.85%	6.08%	3.77 pp
30	28.18%	10.09%	6.22%	3.87 pp
50	28.53%	10.27%	6.29%	3.98 pp
80	28.63%	10.32%	6.32%	4.00 pp
100	28.66%	10.34%	6.33%	4.01 pp

M0 reproduces the baseline procedure of Section 4.3, $H_0(e) = \text{dom}(A(e))$. M3 and M4 match the identically labeled rows of Table 5: M3 adds manifest-level scope evidence and yields the strict upper bound, M4 further removes extensions explainable by a legitimate dynamic-access mechanism and yields the loose lower bound. Interval width is the M3 minus M4 gap in percentage points.

at $t = 10$. It reaches 4.01 points at $t = 100$, a total growth of 0.54 points across the entire sweep. Three quarters of that growth happens before $t = 30$: the width moves from 3.47 to 3.87 points between $t = 10$ and $t = 30$, then crawls from 3.87 to 4.01 points across the remaining range up to $t = 100$.

This pattern traces back to a simple fact about the underlying data. Most extensions that carry manifest-level or dynamic-access evidence touch a narrow set of domains, well under 20 in the great majority of cases. Raising t beyond that range recovers only a thin residual tail on both the upper-bound and lower-bound sides. The two bounds move together because they draw from the same domain-count distribution; neither one runs away from the other as t grows.

We retain $t = 10$ as the anchor. It sits inside the flat region for both the baseline rate and the revised interval. The interval reported at this

threshold, 5.68% to 9.15%, would shift by less than a single percentage point on either bound under any threshold from 15 through 100. A different nearby choice of t would not have changed the substantive finding of Section 4.3.2.

5.3.4. Category-level analysis of host permission mismatch

We repeated the host-permission analysis at the category level under the procedure of Section 4.3. Each of the 129,953 extensions maps to one primary CWS category via `c_extension_category`. 719 extensions carry more than one category label. For these we keep the category with the smallest category_id. The mapping covers 17 named categories plus 25 extensions with no category label, grouped as Unknown/Uncategorized.

The procedure yields an interval, not a point estimate. For every category and every threshold we report two numbers: the M4 lower bound (flagged only) and the M3 upper bound (flagged plus undetermined), following the convention of Table 7. Per-category counts sum exactly to this script's own global total at every threshold tested (5, 10, 15, 20). The check produced zero discrepancy in all four cases.

Table 8 reports this interval for all 18 groups. productivity/workflow (43,652), productivity/tools (27,049), and productivity/developer (10,435) together hold 81,136 extensions, 62.4% of the dataset.

At $t = 10$, lifestyle/games ranks lowest, at 0.83%–1.00% ($N = 3601$). Three categories rank highest, but they do not keep the same order across bounds. Rank them by the M4 lower bound and productivity/developer wins, at 11.41%, ahead of make_chrome_yours/privacy at 9.41% and make_chrome_yours/accessibility at 7.96%. Rank them by the M3 upper bound instead and make_chrome_yours/privacy takes the lead, at 18.27%, ahead of productivity/developer at 17.85%. Accessibility holds third place under both bounds.

Table 9 rescales every category against the global denominator, $n = 129,953$, so rows sum to the global total at every threshold.

At $t = 10$, productivity/workflow contributes 2.19%–3.35% of the global 5.68%–9.15% interval. That is the largest single share. productivity/tools contributes 0.90%–1.80%. productivity/developer contributes 0.92%–1.43%. These three categories together account for 4.01%–6.58% of the global interval: 70.6% of the lower bound and 71.9% of the upper bound. No other category's contribution exceeds 0.41% on the lower bound or 0.57% on the upper bound. lifestyle/social holds both of those maxima.

This concentration tracks category size, not category rate. productivity/workflow holds 33.6% of all extensions, yet its own rate, 6.53%–9.98%, sits below the two smaller categories that carry the highest per-category rates. Two categories clear the global range entirely at $t = 10$: productivity/developer (11.41%–17.85%) and make_chrome_yours/privacy (9.41%–18.27%), both above the global upper bound of 9.15%. A third, make_chrome_yours/accessibility (7.96%–11.32%), only partly clears it. Its lower bound falls inside the 5.68%–9.15% interval; only its upper bound sits outside. Four categories sit entirely below the global range, not one: lifestyle/games (0.83%–1.00%), productivity/education (1.30%–4.69%), lifestyle/well_being (1.51%–4.79%), and lifestyle/travel (2.79%–3.48%). Every one of their upper

Table 8

Mismatch-flagged interval within each CWS category under four domain-count thresholds, computed with the revised procedure of Section 4.3. Each cell reports the M4 lower bound and the M3 upper bound as *lower%–upper%*.

Category	N	Th = 5	Th = 10	Th = 15	Th = 20
Productivity/workflow	43,652	5.78%–8.77%	6.53%–9.98%	6.93%–10.60%	7.13%–10.93%
Productivity/tools	27,049	3.70%–7.28%	4.34%–8.64%	4.46%–8.99%	4.62%–9.25%
Productivity/developer	10,435	10.94%–16.73%	11.41%–17.85%	11.64%–18.23%	11.70%–18.30%
lifestyle/social	9,201	4.91%–6.51%	5.83%–8.03%	6.04%–8.42%	6.12%–8.58%
lifestyle/shopping	8,753	2.52%–4.48%	3.77%–6.19%	4.59%–7.13%	4.74%–7.53%
lifestyle/fun	6,989	4.42%–5.65%	4.92%–6.21%	5.02%–6.34%	5.09%–6.42%
make_chrome_yours/accessibility	6,219	7.16%–10.31%	7.96%–11.32%	8.09%–11.58%	8.15%–11.75%
lifestyle/games	3,601	0.44%–0.56%	0.83%–1.00%	0.86%–1.03%	0.86%–1.17%
Productivity/communication	2,610	2.41%–3.75%	2.87%–6.09%	2.95%–6.28%	2.95%–6.40%
Productivity/education	2,538	1.22%–4.10%	1.30%–4.69%	1.54%–6.34%	1.85%–6.66%
make_chrome_yours/functionality	2,194	3.14%–6.02%	4.15%–7.29%	4.24%–7.52%	4.51%–7.79%
lifestyle/entertainment	2,005	2.19%–5.74%	2.39%–6.28%	2.39%–6.63%	2.39%–6.63%
make_chrome_yours/privacy	1,467	8.04%–16.16%	9.41%–18.27%	10.09%–19.09%	10.22%–19.22%
lifestyle/news	1,375	5.09%–5.89%	5.38%–6.25%	5.53%–6.62%	5.60%–6.69%
lifestyle/art	1,156	4.50%–7.27%	4.84%–7.79%	4.93%–7.87%	4.93%–7.87%
lifestyle/well_being	397	1.51%–4.79%	1.51%–4.79%	1.51%–4.79%	1.51%–4.79%
lifestyle/travel	287	0.70%–0.70%	2.79%–3.48%	2.79%–4.53%	2.79%–4.53%
Unknown/Uncategorized	25	12.00%–12.00%	12.00%–12.00%	12.00%–12.00%	12.00%–12.00%

N = extensions in this category among the 129,953 filtered extensions. Each cell reports the M4 lower bound and the M3 upper bound. **Unknown/Uncategorized** has $N = 25$.

Table 9

Each CWS category's contribution to the global mismatch-flagged interval under the revised procedure, against the global denominator ($n = 129,953$). Each cell reports the M4 lower bound and M3 upper bound as *lower%–upper%*.

Category	N	Th = 5	Th = 10	Th = 15	Th = 20
Productivity/workflow	43,652	1.94%–2.95%	2.19%–3.35%	2.33%–3.56%	2.40%–3.67%
Productivity/tools	27,049	0.77%–1.52%	0.90%–1.80%	0.93%–1.87%	0.96%–1.92%
Productivity/developer	10,435	0.88%–1.34%	0.92%–1.43%	0.93%–1.46%	0.94%–1.47%
lifestyle/social	9,201	0.35%–0.46%	0.41%–0.57%	0.43%–0.60%	0.43%–0.61%
lifestyle/shopping	8,753	0.17%–0.30%	0.25%–0.42%	0.31%–0.48%	0.32%–0.51%
lifestyle/fun	6,989	0.24%–0.30%	0.26%–0.33%	0.27%–0.34%	0.27%–0.35%
make_chrome_yours/accessibility	6,219	0.34%–0.49%	0.38%–0.54%	0.39%–0.55%	0.39%–0.56%
lifestyle/games	3,601	0.01%–0.02%	0.02%–0.03%	0.02%–0.03%	0.02%–0.03%
Productivity/communication	2,610	0.05%–0.08%	0.06%–0.12%	0.06%–0.13%	0.06%–0.13%
Productivity/education	2,538	0.02%–0.08%	0.03%–0.09%	0.03%–0.12%	0.04%–0.13%
make_chrome_yours/functionality	2,194	0.05%–0.10%	0.07%–0.12%	0.07%–0.13%	0.08%–0.13%
lifestyle/entertainment	2,005	0.03%–0.09%	0.04%–0.10%	0.04%–0.10%	0.04%–0.10%
make_chrome_yours/privacy	1,467	0.09%–0.18%	0.11%–0.21%	0.11%–0.22%	0.12%–0.22%
lifestyle/news	1,375	0.05%–0.06%	0.06%–0.07%	0.06%–0.07%	0.06%–0.07%
lifestyle/art	1,156	0.04%–0.06%	0.04%–0.07%	0.04%–0.07%	0.04%–0.07%
lifestyle/well_being	397	0.00%–0.01%	0.00%–0.01%	0.00%–0.01%	0.00%–0.01%
lifestyle/travel	287	0.00%–0.00%	0.01%–0.01%	0.01%–0.01%	0.01%–0.01%
Unknown/Uncategorized	25	0.00%–0.00%	0.00%–0.00%	0.00%–0.00%	0.00%–0.00%
Global total	129,953	4.98%–7.95%	5.68%–9.15%	5.94%–9.61%	6.08%–9.85%

Table 10

Mismatch-flagged rate by manifest version.

Manifest version	Total	Flagged	Rate
Manifest V2	59,178	21,669	36.62%
Manifest V3	139,880	76,403	54.62%
Overall	199,058	98,072	49.3%

bounds sits under the global lower bound of 5.68%. The procedure still supports the same conclusion as before: no single category drives the global rate. But the outlier list runs longer on both ends than a single category, and the category that leads the pack switches between productivity/developer and make_chrome_yours/privacy depending on which bound gets read.

5.3.5. MV2 vs. MV3 mismatch

To test whether the Manifest V3 transition lowers the mismatch-flagged rate, we partitioned the dataset by manifest version and computed the mismatch-flagged rate independently for each group. Of the 199,058 extensions analyzed, 139,880 (70.3%) declare Manifest V3 and 59,178 (29.7%) declare Manifest V2. Table 10 reports the mismatch-flagged counts and rates for each group.

The result is counter-intuitive. MV3 extensions are mismatch-flagged at a rate of 54.62%, which is 18 percentage points higher than the 36.62% rate observed for MV2 extensions. This finding does not imply that MV3 worsens permission hygiene by design. Rather, it reflects two confounding factors.

First, MV3 adoption is skewed toward recently published extensions. Newer extensions are more likely to follow boilerplate templates that include a broad default permission set. Older MV2 extensions have had more time for community review and permission trimming.

Second, MV3 introduced scripting as a replacement for the MV2 pattern of `activeTab` combined with `tabs.executeScript`. Table 2 shows that scripting accounts for the second-largest share of mismatches (34,611 instances, 17.51% of all recorded mismatches.) Many MV3 extensions declare scripting as a precaution. CPDetector detects no matching `chrome.scripting` API call in these extensions. This pattern did not appear under MV2.

Overall, the grouping results confirm that API changes at the platform level do not automatically lower the mismatch-flagged rate. The persistence of unmatched permission declarations, regardless of manifest version, is consistent with developers declaring permissions speculatively rather than strictly according to detected code usage. The mismatch-flagged rate for MV3 extensions reached 54.62%. This shows

the pattern has not improved across manifest-version iterations. Furthermore, the MV3 migration is associated with new patterns of statically unmatched permission declarations, particularly around permissions such as `scripting`.

5.3.6. Complexity analysis

The component Declared Permission Extractor parses one manifest file per extension and branches on manifest version. Manifest V2 stores API and host permissions in one field. Manifest V3 splits them into two. Per-extension cost reaches $O(M)$, where M counts manifest bytes. This figure depends on that extension's own file, never on the other $n - 1$ extensions in the corpus. Total cost across the corpus reaches $O(nM)$, and n enters this bound directly, not as a hidden constant. Manifest files run a few kilobytes. Our corpus held $n = 199,058$ extensions, and the bound stayed linear, never quadratic, across that entire count.

Time cost of Used Permission Extractor grows as $O(n\bar{N}K)$, and space cost stays flat at $O(K)$ plus one extension's identifier list at a time. Here n counts extensions, $\bar{N}$ averages identifiers extracted per extension, and K fixes the size of the reference API list that every identifier gets checked against. Each extension holds its own identifier list in memory, then releases it once processing finishes, so memory never accumulates across the corpus.

Permission-API mismatch detection costs $O(n)$ time and $O(n)$ space, and n enters both bounds directly. Host detection costs $O(|H_O(e)|)$ per extension, where $H_O(e)$ counts the distinct domains an extension actually contacts. The cost therefore scales with the size of this set for each extension. It is not a fixed corpus-wide constant. Table 6 reports an average of two and a maximum of nine, but that figure covers only the source-level count $|\text{dom}(A(e)) \setminus \{\star\}|$ for the 33,998 extensions flagged under the baseline procedure (M0, $t = 10$). It says nothing about the other 95,955 extensions in the 129,953-extension host-permission sample, and nothing about the full 199,058-extension corpus. The per-extension cost $O(|H_O(e)|)$ therefore varies extension by extension, tracking however many domains that extension's manifest and source code together expose.

API detection costs $O(PA)$ per extension, where P counts declared permissions and A counts the APIs an extension actually calls. This product never exceeds $O(K^2)$, where K caps the predefined API universe. Two dictionaries hold every extension's permissions and API calls before the check runs, and that sets peak memory at $O(n)$, not higher. Flagging held at $n = 199,058$, 98,072 extensions carried a mismatch, and those extensions logged 197,646 mismatches in total, two per extension on average. This confirms the bound stays linear at web-store scale rather than degrading as the corpus grows.

5.3.7. Manual audit

To ground the 49.3% figure in ground-truth evidence, we conducted two independent manual audits on disjoint samples drawn from the corpus. Audit A and Audit B test two different sets of extensions. Audit A draws its 150 extensions from the 98,072 extensions CPDetector labeled as mismatch-flagged. Audit B draws its 150 extensions from the extensions CPDetector left unflagged. Together the two audits cover both sides of CPDetector's binary output: flagged and unflagged.

Audit A sampled 150 extensions from the 98,072 flagged cases. Each extension was audited by reading its full JavaScript source and verifying whether every unmatched permission lacked a genuine API call. 139 were confirmed true positives and 11 were false positives. Precision on the flagged set is 92.7% (139/150).

Audit B independently sampled 150 extensions from the set not flagged by CPDetector as mismatched. Within this unflagged set, CPDetector further classifies each extension by whether it detected any API call among the extension's permissions: 109 extensions were classified as having detected APIs, and 41 were classified as having no detected APIs. Manual inspection of full JavaScript source confirmed 103 of the 109 as genuine detections (TP_{api}) and found 6 to be spurious (FP_{api}). Of the 41 classified as having no APIs, 40 were confirmed correct (TN_{api})

Table 11

Extension-level mismatch classification results for CPDetector, pooling Audit A ($n = 150$) and Audit B ($n = 150$).

	Audit A	Audit B	Pooled ($n = 300$)
True Positive (TP)	139	0	139
False Positive (FP)	11	0	11
True Negative (TN)	0	143	143
False Negative (FN)	0	7	7

and 1 was found to actually contain a genuine but undetected API call (FN_{api}). Here the positive class is "an API call is genuinely present". These four counts evaluate only the API-detection step, not CPDetector's mismatch verdict. This yields, for CPDetector's API detection step evaluated on the unflagged set, an accuracy of 95.3% (143/150), a precision of 94.5% (103/109), a recall of 99.0% (103/104), and an F_1 of 96.7%. At the finer-grained API-entry level, 346 of 353 individual API-call records (98.0%) were confirmed as genuine calls.

The 11 false positives in Audit A stem from one root limitation of the *Used Permission Extractor*: the token-adjacency matcher requires the literal object name to appear as the immediately preceding token in the identifier sequence. Two common coding patterns defeat this check. First, extensions that alias the Chrome API object to a local variable for cross-browser compatibility replace the expected object name with an alias, so the required token is absent. Second, minified or obfuscated bundles rename all identifiers to short substitutes, eliminating the original object name from the token stream entirely. In both cases, genuine API calls exist in the source but produce no match, causing declared permissions to be flagged as unused. The 6 false positives in Audit B arise from the opposite direction: the token extractor operates on raw source text without distinguishing executable statements from comments and string literals, so API names embedded in comments or string constants are recorded as genuine calls.

Table 11 pools Audit A and Audit B into a single 300-extension evaluation of CPDetector's extension-level mismatch classification. The positive class is defined as genuinely mismatch-flagged, confirmed by reading the full JavaScript source. True positive (TP) counts an extension CPDetector flagged and the audit confirmed as genuinely mismatched. False positive (FP) counts an extension CPDetector flagged that the audit found was not genuinely mismatched. True negative (TN) counts an extension CPDetector left unflagged that the audit confirmed was genuinely not mismatched. False negative (FN) counts an extension CPDetector left unflagged that the audit found was genuinely mismatched. Audit A supplies TP and FP directly. Audit B supplies TN and FN once its four relabeled outcomes are folded into the mismatch verdict as described above.

Table 11 gives four metrics for CPDetector's mismatch classification. Accuracy equals $(TP+TN)/300$, or $(139+143)/300$, or 94.0%. Precision equals $TP/(TP+FP)$, or $139/150$, or 92.7%, the same figure reported for Audit A alone, since Audit A alone determines this ratio. Recall equals $TP/(TP+FN)$, or $139/146$, or 95.2%. The F_1 score equals $2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$, or 93.9%.

6. PermissionGuard

Due to the lack of granularity in Chrome's current permission management, users cannot clearly understand the risk levels of different permissions or recognize the potential threats posed by extensions flagged with a permission-API mismatch. Therefore, developing a Chrome extension that enables fine-grained permission control is essential. It can help users better understand and defend against the risks associated with malicious extensions.

6.1. Design of new permission warning mechanism

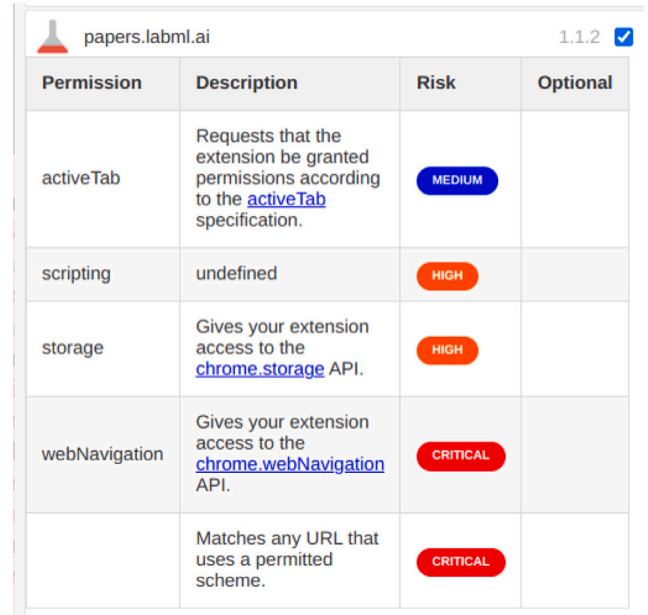
We redesigned the permission management mechanism of Google based on three classification criteria: the broadness of permissions, the likelihood of triggering warnings, and the potential risks associated with permission granting. The permissions of Chrome extensions were categorized into four levels: Critical, High, Medium, and Low. Extensions whose permissions exhibit issues in all three criteria are labeled as Critical. Extensions with issues in two of the criteria are labeled as High. Extensions with issues in one criterion are labeled as Medium. Extensions without issues in any of the three criteria are labeled as Low. A detailed description of the three classification criteria is provided below.

- Permissions are evaluated based on whether they directly relate to user privacy. If a permission inherently involves access to privacy-sensitive data or user behavior, it is flagged as a high-risk indicator. For example, granting the `history` permission allows the extension to read and modify the user's entire browsing history, which directly exposes personal browsing habits and visited websites. Such permissions are marked with a value of 1 in the assessment criteria.
- The potential for Chrome extension permissions to trigger warnings was examined. Permissions that trigger warnings are often associated with sensitive or potentially risky operations, which may involve the collection of user privacy data. Therefore, this criterion was established as a standard. The list of permissions that trigger warnings is presented in Table 12.
- The potential risks associated with permission granting behaviors were assessed, focusing primarily on the methods of acquisition and timing of use. Certain permissions may be granted without clear notification to the user or may be automatically acquired without explicit user consent. Such granting behaviors are considered risky. For example, the `<all_urls>` permission, though categorized under `host_permissions`, allows an extension to access any webpage. Since users are not notified during installation, the risk of this permission is relatively high. In contrast, the `power` permission, which enables the extension to monitor the device's power state and adjust its behavior accordingly, does not introduce significant risks. It merely monitors the power state of the device without posing potential danger.

We developed an extension called *PermissionGuard* by leveraging the newly designed permission warning mechanism in conjunction with the mismatch detection introduced above. This extension flags any permission-API mismatch in installed extensions and reports a risk level for each permission. This helps users recognize whether an installed extension may be carrying more permissions than its detected behavior warrants.

6.2. Specific design mechanisms

Currently, the permission warning mechanism of Google extensions mainly determines warnings based on the invasiveness of permissions. However, when a user installs an extension, only the permissions that trigger a warning are displayed. Some permissions that do not trigger warnings may still be used for malicious activities. For example, the `cookies` permission does not trigger a warning under the existing Google warning mechanism, yet many malicious actions often rely on this permission. Malicious behaviors aimed at stealing sensitive information typically exploit the `cookies` permission to steal users' cookie data. Therefore, we reclassified the extended permissions based on whether the permission directly involves user privacy, the risk of the permission granting operation, and whether it triggers a warning. The specific classification is shown in Table 12. A value of 0 denotes the absence, and 1 denotes the presence of the respective attribute.



Permission	Description	Risk	Optional
activeTab	Requests that the extension be granted permissions according to the activeTab specification.	MEDIUM	
scripting	undefined	HIGH	
storage	Gives your extension access to the chrome.storage API.	HIGH	
webNavigation	Gives your extension access to the chrome.webNavigation API.	CRITICAL	
	Matches any URL that uses a permitted scheme.	CRITICAL	

Fig. 4. PermissionGuard warns users about permissions of installed extensions.

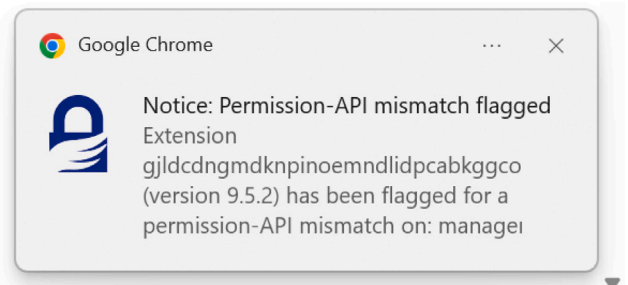


Fig. 5. PermissionGuard alerts users that the installed extension may have permissions exceeding detected usage.

6.3. Results of PermissionGuard

Due to the limitations of the existing permission warning mechanism, users face difficulty in intuitively identifying the potential security implications of an extension's permissions. Moreover, there is insufficient awareness regarding the risk levels of various permissions. To address this issue, we developed a Google Chrome extension named *PermissionGuard*. This extension displays the risk levels of the permissions granted to extensions installed in the browser and flags any permissions with a potential mismatch. This helps users better understand the potential malicious actions an extension may carry. For example, *PermissionGuard* reports the risk level and any detected permission-API mismatch for the "papers.labml.ai" extension installed by the user. The result of the analysis is shown in Fig. 4. Furthermore, *PermissionGuard* displays notifications regarding permissions flagged as potential mismatches in the extensions installed, as shown in Fig. 5.

This extension aims to display permission details and any flagged mismatches for the extensions installed by the user, helping them understand how permissions are used across extensions. Through this classification, users can view the risk levels of all extension permissions and identify any permissions flagged as potential mismatches. It allows users to review the permissions of enabled, disabled, and all extensions. Additionally, users can manage their extensions by enabling or disabling specific ones.

Table 12
Classification of permissions in the new warning mechanism for permissions.

Permission	Privacy	Warning	Risk	Classification	Permission	Privacy	Warning	Risk	Classification
accessibilityFeatures.modify	0	1	1	High	accessibilityFeatures.read	0	1	1	High
activeTab	0	0	1	Medium	alarms	0	0	0	Low
audio	0	0	0	Low	audioCapture	1	0	1	High
background	0	0	0	Low	bookmarks	1	1	0	High
browsingData	1	0	0	Medium	certificateProvider	0	0	1	Medium
clipboard	0	0	0	Low	clipboardRead	1	1	1	Critical
clipboardWrite	0	1	0	Medium	contentSettings	1	1	1	Critical
contextMenus	0	0	0	Low	cookies	1	0	1	High
debugger	1	0	1	High	declarativeContent	0	0	0	Low
declarativeNetRequest	0	1	1	High	declarativeNetRequestWithHostAccess	0	0	1	Medium
declarativeNetRequestFeedback	0	1	1	High	desktopCapture	0	1	1	High
dns	1	0	1	High	documentScan	1	0	0	Medium
downloads	1	1	1	Critical	downloads.open	1	1	1	Critical
downloads.ui	0	1	0	Medium	enterprise.deviceAttributes	1	0	0	Medium
enterprise.hardwarePlatform	1	0	0	Medium	enterprise.networkingAttributes	1	0	0	Medium
enterprise.platformKeys	1	0	0	Medium	favicon	0	1	0	Medium
fileBrowserHandler	0	0	0	Low	fileSystemProvider	1	0	1	High
fontSettings	1	0	0	Medium	gcm	1	0	0	Medium
geolocation	1	1	1	Critical	history	1	1	1	Critical
identity	1	0	1	High	identity.email	1	1	1	Critical
idle	0	0	0	Low	loginState	0	0	0	Low
management	0	1	1	High	nativeMessaging	0	1	1	High
notifications	0	1	0	Medium	offscreen	0	0	0	Low
pageCapture	1	1	1	Critical	platformKeys	0	0	1	Medium
power	0	0	0	Low	printerProvider	0	0	0	Low
printing	0	0	0	Low	printingMetrics	0	0	0	Low
privacy	1	1	1	Critical	processes	0	0	1	Medium
proxy	1	1	0	High	readingList	0	1	0	Medium
runtime	0	0	0	Low	scripting	1	0	1	High
search	0	0	0	Low	sessions	1	1	1	Critical
sidePanel	0	0	0	Low	storage	1	0	1	High
system.cpu	0	0	0	Low	system.display	0	0	0	Low
system.memory	0	0	0	Low	system.storage	1	1	0	High
tabCapture	1	1	1	Critical	tabGroups	0	1	0	Medium
tabs	1	1	1	Critical	topSites	1	1	0	High
tts	0	0	0	Low	ttsEngine	0	1	0	Medium
unlimitedStorage	1	0	0	Medium	vpnProvider	1	0	1	High
wallpaper	0	0	0	Low	webAuthenticationProxy	1	1	0	High
webNavigation	1	1	1	Critical	webRequest	1	0	1	High
webRequestBlocking	1	0	0	Medium	<all_urls>	1	1	1	Critical
https://*/*	1	0	1	High	http://*/*	1	0	1	High
:///*	1	1	1	Critical	file://*/*	1	0	1	High

7. Related work

7.1. Malicious extension detection and vulnerability identification

Malicious behavior detection in browser extensions is generally categorized into static analysis and dynamic analysis. Relevant studies on dynamic analysis include methods such as “Excision”, “WebEval”, and “Hulk”. A browser-based method named “Excision” was proposed by Arshad et al. (2017) for the automatic detection and blocking of malicious third-party content during webpage loading or browser extension execution. Experimental results demonstrated a detection rate of 93.39% and a false positive rate of only 0.59%. Jagpal et al. (2015) proposed and implemented WebEval, the first efficient system for detecting malicious browser extensions, achieving a detection rate of 96.5%. A dynamic analysis system named “Hulk” was proposed by Kapravelos et al. (2014) to detect malicious behaviors by monitoring execution activities. An analysis of 48,332 Chrome extensions identified 130 as malicious and 4712 as suspicious. Hulk uses a combination of permission declarations and runtime API calls to detect abnormal behavior. However, its goal is to distinguish between malicious and benign extensions. CPDetector addresses a completely different issue: inconsistencies between permission declarations and actual usage.

The methods for performing static analysis on malicious extensions are described as follows. Pantelaios et al. (2020) proposed a novel method for detecting malicious Chrome extensions based on update deltas. This approach successfully identified 21 clusters of malicious extensions and 143 individual malicious extensions, 44% of which had

not been flagged by users or the CWS. Additionally, some researchers have employed a combination of dynamic and static analysis to detect malicious browser extensions. Wang et al. (2018) proposed a machine learning-based approach integrating static and dynamic analysis for this purpose. Experimental results demonstrated that the proposed method achieved accuracy rates of 96.52% and 95.18%.

However, these approaches assume that extensions are either benign or malicious. This paper addresses a more widespread and subtle issue: permission-API mismatch. Whether benign or malicious, extensions may declare permissions for which no corresponding usage is detected in their source code, thereby expanding the attack surface without triggering existing detection mechanisms.

Fass et al. (2021) proposed DoubleX, a static analysis framework that constructs data-flow graphs to identify suspicious inter-extension communication patterns and potential exfiltration channels. While DoubleX excels at identifying structural vulnerabilities in extension code, it does not address whether the permissions declared by an extension are proportionate to its actual API usage.

Supply-chain attacks and malicious update patterns amplify the risk posed by unused permissions. Pantelaios et al. (2020) observed that malicious behavior in extensions typically surfaces *after* a user base has formed. The 44% of the 143 flagged extensions had never been reported before analysis, suggesting dormant permissions were activated only once the extension reached scale. The PDF Toolbox case in Section 3 follows the same pattern. The extension kept its permission declaration unchanged for 18 months while activating malicious logic only after delay. An attacker who compromises a developer account or

a third-party dependency inherits the full declared permission set of the original extension. Every unused permission in that manifest becomes immediately callable without triggering a new user prompt, setting the ceiling on what the compromise can reach.

7.2. Permission-API mismatch analysis

The closest related work is by [Picazo-Sanchez et al. \(2022\)](#), who analyzed 170,927 extensions, including apps, browser extensions, and themes, in December 2019. This study employed association rule mining to identify discrepancies between declared and actual permissions. Their headline figure, 60.2% mismatch-flagged, spans the full mixed corpus. Restricted to browser extensions alone, 136,018 of them, their reported rate is 59.2%, the figure most comparable to this paper's scope.

Our work CPDetector extends and improves upon this foundation in three significant respects. First, our dataset of 199,058 extensions (2024–2025) is the first large-scale permission-API mismatch study conducted after mandatory Manifest V3 adoption. This enables longitudinal assessment of whether this platform-level security intervention has materially improved permission hygiene in practice.

Second, while Picazo-Sanchez folded host-permission scope into a single mismatch-flagged verdict alongside API permissions, CPDetector introduces dual-dimensional detection: a dedicated three-criterion procedure evaluates host-permission scope independently of API-permission matching. Third, CPDetector produces a four-level risk classification (*Critical*, *High*, *Medium*, *Low*) based on privacy sensitivity, warning triggers, and granting risk, enabling actionable triage rather than a binary verdict. The accompanying *PermissionGuard* browser extension delivers this classification to end users in real time.

[Heule et al. \(2015\)](#) proposed a theoretical MAC-based architecture for least-privilege enforcement of browser extensions, demonstrating that a principled permission model could substantially reduce extension capabilities without functional degradation. However, their work remained a theoretical proposal without large-scale empirical validation or deployment. [Liu et al. \(2025\)](#) investigated code-strategy consistency in mini-programs. Their work shares conceptual similarities with the CPDetector methodology but applies to a different platform. Their findings resonate with this paper's observations: over-declaration constitutes a systemic developer behavior across ecosystems. However, Chrome extensions differ fundamentally from these ecosystems in terms of their code structure and permission architecture. Detecting excessive permissions requires a redesign of the approach.

The browser platform has introduced several mitigations that partially address permission-API mismatch. Site isolation assigns each origin a dedicated renderer process, bounding the damage a compromised content script can cause ([Reis et al., 2019](#)). Content Security Policy (CSP) restricts which scripts a page may execute, limiting data exfiltration channels. Neither mechanism constrains what permissions an extension declares at install time. Within the Chromium community, proposals have been put forward for source-based permission binding and permission scope restrictions to address this gap. For example, binding permissions for sensitive APIs such as cookies to specific domains rather than applying them globally. Another example is allowing users to revoke permission for a specific API individually after installation. However, neither of these measures has been implemented yet ([Chrome Extensions, 2024](#)). CPDetector's finding that 47.5% of mismatch-flagged extensions declare `<all_urls>`, while the source-level scan detected on average only two domains for these extensions. It provides empirical justification for these proposals. It demonstrates that the gap between permissions and actual usage is measurable and persistent, and has not been addressed by existing platform safeguards.

Analogous measurement work on other platforms provides broader context. [Felt et al. \(2011\)](#) audited Android apps and found that 35.8% of them held additional permissions, which is directly analogous to the structure of Chrome extensions. In 2019, [Picazo-Sanchez et al. \(2022\)](#)

analyzed 136,018 Chrome browser extensions and found, using AST-based static analysis, that 40.8% were compliant with the spirit of least privilege. This leaves 59.2% mismatch-flagged. Our results show an overall mismatch rate of 49.3% for 2024–2025. A version-stratified breakdown reveals that MV3 extensions, 139,880 of them, 70.3% of the dataset, are mismatch-flagged at 54.62%, compared to 36.62% for MV2 extensions, 59,178 of them, 29.7% of the dataset. The MV3 transition did not lower the mismatch-flagged rate. A new pattern also appears after the transition: many MV3 extensions declare `scripting`, and CPDetector detects no matching `chrome.scripting` call in these extensions. Platform-level API changes alone do not correct developer permission behavior.

8. Limitations

The proposed CPDetector exhibits certain limitations. First, its design relies on static analysis. This approach compares permissions declared in the `manifest.json` file with detected API calls to identify mismatch-flagged extensions. However, static analysis cannot fully or accurately determine JavaScript code's actual runtime behavior. Due to JavaScript's highly dynamic nature, static analysis cannot cover all possible execution paths. It inevitably misses some API calls. Example includes calls using dynamic string concatenation. Consequently, CPDetector may misclassify an extension as *mismatch-flagged* if it genuinely uses a permission but the corresponding API call escapes static detection. Second, extension developers may intentionally obfuscate their code. This makes API calls difficult for static analysis tools to recognize, thereby evading detection and hiding actual permission usage.

Dynamic analysis offers an alternative, but it carries its own constraints when applied to permission-API mismatch detection at this scale. First, triggering coverage stays incomplete. A dynamic system observes only the code paths an extension executes during the test window. A permission tied to a rarely triggered event handler, or gated behind a remote configuration flag, may never fire during monitored execution. The system then misses the very API call it needs to confirm legitimate use. Second, environment dependence limits reach. Extension behavior can vary across operating systems, browser versions, and network conditions. Replicating this matrix across 199,058 extensions demands execution infrastructure far beyond what static parsing requires. Third, evasion remains a known weakness. An extension can probe its execution context for signs of monitoring, such as synthetic page elements or sandboxed network responses, and suppress its behavior accordingly. Static analysis avoids all three constraints by reading the full source artifact directly, without depending on any particular runtime path being exercised. This property makes static analysis the more practical choice for a corpus at web-store scale, even though it introduces the false-positive and false-negative risks discussed above.

9. Conclusion

We propose CPDetector, a static triage system that flags permission-API mismatches in Chrome extensions. The CPDetector operates in three phases: data collection, information extraction, and permission-API mismatch detection. Additionally, we have designed a novel Google Permission alert mechanism and developed a Chrome extension, named *PermissionGuard*. It alerts users about the potential risks associated with the permissions requested by installed extensions. It flags permissions that extend beyond the API usage CPDetector detects in the extension's source code. We conducted a large-scale study on 199,058 extensions collected from the CWS using the CPDetector. Our findings showed that 98,072 extensions were flagged as permission-API mismatch candidates. 5.68%–9.15% of extensions were flagged as host-permission mismatch candidates under the revised three-criterion procedure. These results show that a large share of browser extensions carry a gap between declared permissions and detected API usage.

Table A.13

Permissions and corresponding extension APIs in Manifest V2.

No.	Permission	Extension API	No.	Permission	Extension API
1	accessibilityFeatures.modify	chrome.accessibilityFeatures	2	accessibilityFeatures.read	chrome.accessibilityFeatures
3	activeTab	chrome.tabs.executeScript, chrome.tabs.insertCSS	4	alarms	chrome.alarms
5	audio	chrome.audio	6	bookmarks	chrome.bookmarks
7	browsingData	chrome.browsingData	8	certificateProvider	chrome.certificateProvider
9	contentSettings	chrome.contentSettings	10	contextMenus	chrome.contextMenus
11	cookies	chrome.cookies	12	debugger	chrome.debugger
13	declarativeContent	chrome.declarativeContent	14	declarativeWebRequest	chrome.declarativeWebRequest
15	dns	chrome.dns	16	desktopCapture	chrome.desktopCapture
17	documentScan	chrome.documentScan	18	downloads	chrome.downloads
19	downloads.open	chrome.downloads.open	20	downloads.ui	chrome.downloads.setUiOptions
21	enterprise.deviceAttributes	chrome.enterprise.deviceAttributes	22	enterprise.hardwarePlatform	chrome.enterprise.hardwarePlatform
23	enterprise.networkingAttributes	chrome.enterprise.networkingAttributes	24	enterprise.platformKeys	chrome.enterprise.platformKeys
25	fileBrowserHandler	chrome.fileBrowserHandler	26	fileSystemProvider	chrome.fileSystemProvider
27	fontSettings	chrome.fontSettings	28	gcm	chrome.gcm, chrome.instanceID
29	history	chrome.history	30	identity	chrome.identity
31	identity.email	chrome.identity	32	idle	chrome.idle
33	loginState	chrome.loginState	34	management	chrome.management
35	nativeMessaging	chrome.runtime.connectNative, chrome.runtime.sendNativeMessage	36	notifications	chrome.notifications
37	pageCapture	chrome.pageCapture	38	platformKeys	chrome.platformKeys
39	power	chrome.power	40	printerProvider	chrome.printerProvider
41	printing	chrome.printing	42	printingMetrics	chrome.printingMetrics
43	privacy	chrome.privacy	44	processes	chrome.processes
45	proxy	chrome.proxy	46	search	chrome.search
47	sessions	chrome.sessions	48	storage	chrome.storage
49	system.cpu	chrome.system.cpu	50	system.display	chrome.system.display
51	system.memory	chrome.system.memory	52	system.storage	chrome.system.storage
53	tabCapture	chrome.tabCapture	54	tabGroups	chrome.tabGroups
55	tabs	chrome.tabs, chrome.windows	56	topSites	chrome.topSites
57	tts	chrome.tts	58	ttsEngine	chrome.ttsEngine
59	unlimitedStorage	chrome.storage.local	60	vpnProvider	chrome.vpnProvider
61	wallpaper	chrome.wallpaper	62	webNavigation	chrome.webNavigation
63	webRequest	chrome.webRequest	64	webRequestBlocking	chrome.webRequest

CRedit authorship contribution statement

Xiaoyu Cheng: Writing – review & editing, Writing – original draft, Visualization, Software, Project administration, Methodology, Formal analysis, Data curation. **Zhi Wang:** Writing – review & editing. **Wanpeng Li:** Writing – review & editing, Writing – original draft, Supervision, Resources, Data curation. **Wei Zeng:** Writing – review & editing. **Xin Yang:** Data curation.

Code availability

The source code of PermissionGuard is publicly available at <https://github.com/cxy1222/PermissionGuard.git>.

Ethical considerations

This study crawls publicly accessible metadata and CRX packages from the CWS solely for academic research. No user data were collected, no extensions were modified, and all findings were handled in accordance with responsible disclosure practices.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix

See Figs. A.6–A.7 and Tables A.13–A.14.

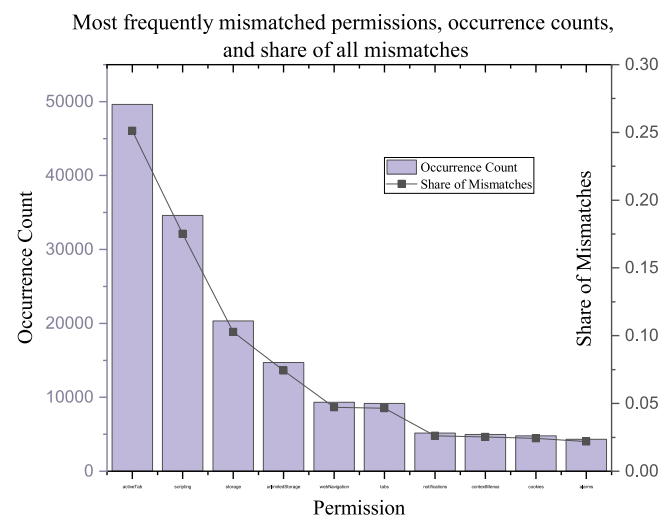


Fig. A.6. Most frequently mismatched permissions, occurrence counts, and share of all mismatches.

Data availability

The code has been open-sourced on GitHub.

Table A.14
Permissions and corresponding extension APIs in Manifest V3.

No.	Permission	Extension API	No.	Permission	Extension API
1	accessibilityFeatures.modify	chrome.accessibilityFeatures	2	accessibilityFeatures.read	chrome.accessibilityFeatures
3	activeTab	chrome.scripting.executeScript, chrome.scripting.insertCSS	4	alarms	chrome.alarms
5	audio	chrome.audio	6	bookmarks	chrome.bookmarks
7	browsingData	chrome.browsingData	8	certificateProvider	chrome.certificateProvider
9	contentSettings	chrome.contentSettings	10	contextMenus	chrome.contextMenus
11	cookies	chrome.cookies	12	debugger	chrome.debugger
13	declarativeContent	chrome.declarativeContent	14	declarativeNetRequest	chrome.declarativeNetRequest
15	declarativeNetRequestWithHostAccess	chrome.declarativeNetRequest	16	declarativeNetRequestFeedback	chrome.declarativeNetRequest
17	dns	chrome.dns	18	desktopCapture	chrome.desktopCapture
19	documentScan	chrome.documentScan	20	downloads	chrome.downloads
21	downloads.open	chrome.downloads.open	22	downloads.ui	chrome.downloads.setUiOptions
23	enterprise.deviceAttributes	chrome.enterprise.deviceAttributes	24	enterprise.hardwarePlatform	chrome.enterprise.hardwarePlatform
25	enterprise.networkingAttributes	chrome.enterprise.networkingAttributes	26	enterprise.platformKeys	chrome.enterprise.platformKeys
27	fileBrowserHandler	chrome.fileBrowserHandler	28	filesystemProvider	chrome.filesystemProvider
29	fontSettings	chrome.fontSettings	30	gcm	chrome.gcm, chrome.instanceID
31	history	chrome.history	32	identity	chrome.identity
33	identity.email	chrome.identity	34	idle	chrome.idle
35	loginState	chrome.loginState	36	management	chrome.management
37	nativeMessaging	chrome.runtime.connectNative, chrome.runtime.sendNativeMessage	38	notifications	chrome.notifications
39	offscreen	chrome.offscreen	40	pageCapture	chrome.pageCapture
41	platformKeys	chrome.platformKeys	42	power	chrome.power
43	printerProvider	chrome.printerProvider	44	printing	chrome.printing
45	printingMetrics	chrome.printingMetrics	46	privacy	chrome.privacy
47	processes	chrome.processes	48	proxy	chrome.proxy
49	readingList	chrome.readingList	50	scripting	chrome.scripting
51	search	chrome.search	52	sessions	chrome.sessions
53	sidePanel	chrome.sidePanel	54	storage	chrome.storage
55	system.cpu	chrome.system.cpu	56	system.display	chrome.system.display
57	system.memory	chrome.system.memory	58	system.storage	chrome.system.storage
59	tabCapture	chrome.tabCapture	60	tabGroups	chrome.tabGroups
61	tabs	chrome.tabs, chrome.windows	62	topSites	chrome.topSites
63	tts	chrome.tts	64	ttsEngine	chrome.ttsEngine
65	unlimitedStorage	chrome.storage.local	66	vpnProvider	chrome.vpnProvider
67	wallpaper	chrome.wallpaper	68	webAuthenticationProxy	chrome.webAuthenticationProxy
69	webNavigation	chrome.webNavigation	70	webRequest	chrome.webRequest

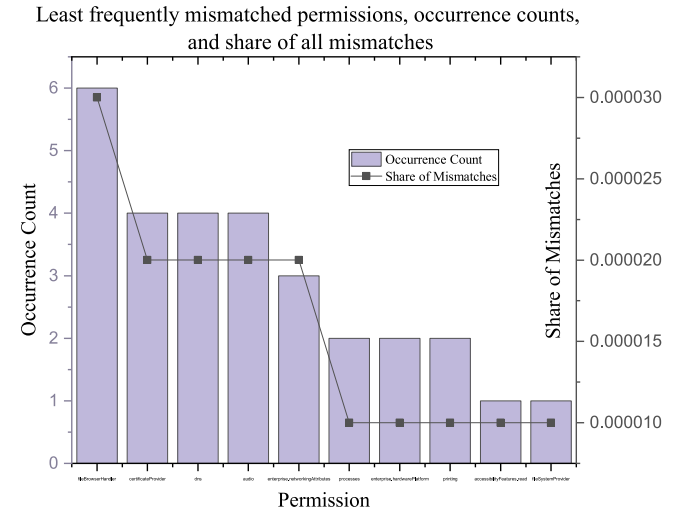


Fig. A.7. Least frequently mismatched permissions, occurrence counts, and share of all mismatches.

References

Arshad, S., Kharraz, A., Robertson, W., 2017. Include me out: In-browser detection of malicious third-party content inclusions. In: Financial Cryptography and Data Security: 20th International Conference, FC 2016, Christ Church, Barbados, February 22–26, 2016, Revised Selected Papers 20. Springer, pp. 441–459.

Barth, A., Felt, A.P., Saxena, P., Boodman, A., 2010. Protecting browsers from extension vulnerabilities. In: Ndss.

Carlini, N., Felt, A.P., Wagner, D., 2012. An evaluation of the google chrome extension security architecture. In: 21st USENIX Security Symposium. USENIX Security 12, pp. 97–111.

Chrome, 2012. Content scripts. URL https://developer.chrome.com/docs/extensions/mv3/content_scripts. (Accessed 07 March 2025).

Chrome, 2023a. Chrome web store review process. URL <https://developer.chrome.com/docs/webstore/review-process#review-time>. (Accessed 15 August 2025).

Chrome, 2023b. Migrating from background pages to service workers. URL https://developer.chrome.com/docs/extensions/mv3/migrating_to_service_workers. (Accessed 07 March 2025).

Chrome, 2025. Grammarly - AI writing and editing assistant. URL <https://chromewebstore.google.com/detail/grammarly-ai-writing-and-kbfmbcaepfbcioakpkpbfkobkghlhen>. (Accessed 16 April 2025).

Chrome Extensions, 2024. Declare permissions in chrome extensions. URL <https://developer.chrome.com/docs/extensions/develop/concepts/declare-permissions>. (Accessed 07 April 2025).

Developers, C., 2012a. Declaring permissions in manifest V2. URL <https://developer.chrome.com/docs/extensions/mv2/declare-permissions>. (Accessed 15 March 2025).

Developers, C., 2012b. Give users options. URL <https://developer.chrome.com/docs/extensions/mv3/options>. (Accessed 07 March 2025).

Developers, G.C., 2024. Chrome extension API: History. URL <https://developer.chrome.com/docs/extensions/reference/api/history>. (Accessed 19 February 2025).

Developers, C., 2024a. Chrome extension APIs reference. URL <https://developer.chrome.com/docs/extensions/reference/api>. (Accessed 15 March 2025).

Developers, C., 2024b. Chrome extension permissions list. URL <https://developer.chrome.com/docs/extensions/reference/permissions-list>. (Accessed 15 March 2025).

Developers, G.C., 2025. Chrome extension cookies API. URL <https://developer.chrome.com/docs/extensions/reference/api/cookies>. (Accessed 19 February 2025).

Fass, A., Somé, D.F., Backes, M., Stock, B., 2021. Doublex: Statically detecting vulnerable data flows in browser extensions at scale. In: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security. pp. 1789–1804.

Felt, A.P., Chin, E., Hanna, S., Song, D., Wagner, D., 2011. Android permissions demystified. In: Proceedings of the 18th ACM Conference on Computer and Communications Security. pp. 627–638.

Google Chrome Developers, 2024. Declare permissions. URL <https://developer.chrome.com/docs/extensions/develop/concepts/declare-permissions>. (Accessed 19 February 2025).

- Heule, S., Rifkin, D., Russo, A., Stefan, D., 2015. The most dangerous code in the browser. In: 15th Workshop on Hot Topics in Operating Systems. HotOS XV.
- Islam, M.M., An Evaluation of Extension Vulnerabilities of Google Chrome.
- Jagpal, N., Dingle, E., Gravel, J.-P., Mavrommatis, P., Provos, N., Rajab, M.A., Thomas, K., 2015. Trends and lessons from three years fighting malicious extensions. In: 24th USENIX Security Symposium. USENIX Security 15, pp. 579–593.
- Kaprauelos, A., Grier, C., Chachra, N., Kruegel, C., Vigna, G., Paxson, V., 2014. Hulk: Eliciting malicious behavior in browser extensions. In: 23rd USENIX Security Symposium. USENIX Security 14, pp. 641–654.
- Liu, L., Mao, J., Lin, Q., Lu, Y., Li, J., Liu, J., 2025. Code to policy consistency detection for mini program based on semantic analysis. *J. Softw.* 36 (11), 5102–5117.
- Liu, L., Zhang, X., Yan, G., Chen, S., et al., 2012. Chrome extensions: Threat analysis and countermeasures. In: NDSS. Citeseer.
- OSCHINA, 2023. Fake Adblock Plus Extension Bypasses Chrome Web Store Review. <https://www.oschina.net/news/89473/fake-adblock-plus-extension>. (Accessed 30 June 2026).
- Palant, W., 2023. Malicious code in PDF toolbox extension. <https://palant.info/2023/05/16/malicious-code-in-pdf-toolbox-extension/>. (Accessed 30 June 2026).
- Pantelaos, N., Nikiforakis, N., Kaprauelos, A., 2020. You've changed: Detecting malicious browser extensions through their update deltas. In: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security. pp. 477–491.
- Picazo-Sanchez, P., Ortiz-Martin, L., Schneider, G., Sabelfeld, A., 2022. Are chrome extensions compliant with the spirit of least privilege? *Int. J. Inf. Secur.* 21 (6), 1283–1297.
- Reis, C., Moshchuk, A., Oskov, N., 2019. Site isolation: Process separation for web sites within the browser. In: 28th USENIX Security Symposium. USENIX Security 19, pp. 1661–1678.
- Sjösten, A., Van Acker, S., Sabelfeld, A., 2017. Discovering browser extensions via web accessible resources. In: Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy. pp. 329–336.
- Somé, D.F., 2019. EmPoWeb: empowering web applications with browser extensions. In: 2019 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 227–245.
- Starov, O., Nikiforakis, N., 2017. Xhound: Quantifying the fingerprintability of browser extensions. In: 2017 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 941–956.
- Tool, E., 2025. Esprima tool. URL <https://esprima.org/demo/parse.html>. (Accessed 19 February 2025).
- Wang, Y., Cai, W., Lyu, P., Shao, W., 2018. A combined static and dynamic analysis approach to detect malicious browser extensions. *Secur. Commun. Netw.* 2018 (1), 7087239.
- Yu, J., Li, S., Zhu, J., Cao, Y., 2023. CoCo: Efficient browser extension vulnerability detection via coverage-guided, concurrent abstract interpretation. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. pp. 2441–2455.